NOTES DE COURS

Notes de Cours sur le logiciel R

Anne PHILIPPE

Nantes Université
Laboratoire de Mathématiques Jean Leray
email : Anne.philippe@univ-nantes.fr
https://philippe-a.github.io/website/

October 7, 2026

Plan I

1. Installer R et RStudio
2. Objets et Opérations
3. Exercices
4. Importer et exporter des données
5. Autour des lois de probabilités
6. Exercices
7. Les graphiques
8. Exercices

Plan II

9. Définir une fonction dans R
10. Exercices
11. Structures de contrôle et Itérations
12. Exercices
13. Outils graphiques en statistique
14. Exercice
15. Le tidyverse
16. Manipuler les donnees avec dplyr

Plan III

17. Mettre en ordre avec tidyr
18. Visualiser avec ggplot2
19. Recoder des variables
20. Inférence statistique
21. Séries Chronologiques

SECTION 1

Installer R et RStudio

R vs RStudio

- **R** : le langage et le moteur de calcul (obligatoire)
- **RStudio** : un IDE (environnement de développement) qui pilote R
  - éditeur de code, console, visualisation des graphiques/variables
  - gestion de projets, intégration Git, R Markdown intégré
- Il faut installer **les deux**, dans cet ordre : R d'abord, RStudio ensuite

Installer R

- Télécharger sur le CRAN : `https://cran.r-project.org`
- **macOS** : choisir le `.pkg` correspondant à la puce (Apple Silicon ou Intel)
- **Windows** : *Download R for Windows* puis *base*
- **Linux** : via le gestionnaire de paquets (`apt`, `dnf`...) ou le CRAN

Vérifier la version installée :

```
R.version.string
```

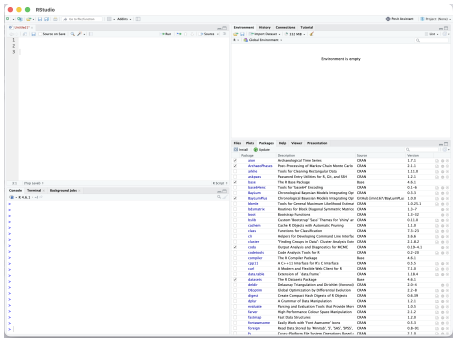

Installer RStudio

- Télécharger RStudio Desktop (gratuit) : `https://posit.co/download/rstudio-desktop/`
- Installer **après** R
- Au premier lancement, RStudio détecte automatiquement l'installation de R existante

Anatomie de RStudio

Quatre panneaux principaux :

- **Source** (haut-gauche) : scripts `.R`, documents `.Rmd`
- **Console** (bas-gauche) : exécution interactive, résultats
- **Environment / History** (haut-droite) : variables en mémoire
- **Files / Plots / Packages / Help** (bas-droite) : navigation, graphiques



Réglages recommandés (RStudio)

`Tools > Global Options` :

- **General** : décocher *Restore .RData into workspace at startup*
- **General** : *Save workspace to .RData on exit* → **Never**
- **Code** > **Saving** : encodage UTF-8 par défaut
- **Appearance** : choisir un thème confortable (police, taille)

Pourquoi ? Repartir d'un environnement propre à chaque session évite les bugs liés à des objets fantômes en mémoire.

Utiliser l'aide

Aide sur une fonction

```
?plot
```

Les démos :

```
# pour obtenir la liste des demos
demo()
demo(graphics)
```

Les exemples:
La fonction `example` exécute les exemples généralement inclus à la fin des fichiers d'aide.

```
example(FUN)
```


Packages

- Toutes les packages ne sont pas chargées au lancement du logiciel
- `library()` retourne la liste des packages installées.
- `library(LIB)` charge la package `LIB`
- `library(help = LIB)` retourne la liste des fonctions de la package `LIB`
- `search(), searchpaths()` retourne la liste des packages chargées.

Savoir lire un message d'erreur

Quatre messages couvrent l'essentiel des blocages en TP.

- `Error: object 'x' not found`
  L'objet n'existe pas : faute de frappe, ou ligne précédente non exécutée. Vérifier avec `ls()`.
- `Error: could not find function "fviz_screeplot"`
  La fonction existe mais son paquet n'est pas chargé : `library(...)`, ou `install.packages(...)` si le paquet est absent.
- `Error: non-numeric argument to binary operator`
  Un calcul sur du texte. Typiquement une colonne importée en `character` : vérifier avec `str()`.
- `Error: subscript out of bounds`
  Indice hors des dimensions : contrôler avec `length()` ou `dim()`.

Le réflexe : lire le message *en entier*, il nomme la fonction et l'objet fautifs. Puis `str(objet)` pour voir ce que l'objet est réellement, et non ce que l'on croit qu'il est.

Avertissement $\neq$ erreur. Un `Warning` n'interrompt pas le calcul, mais signale un résultat possiblement faux : il se lit avec la même attention.

SECTION 2

Objets et Opérations

Opérations élémentaires

1. Opérations élémentaires sur les scalaires : $*$, $-$, $+$, $/$, $\hat{}$

```
2+4
6
```

2. Opérations avec affectation (avec ou sans affichage)

```
x=2+4
x
6
(x=2+4)              # avec affichage du résultat
6
```

3. Les principaux types sont
   - entier , réel, complexe
   - caractère
   - logique : TRUE, FALSE, NA (not available)

Objets

Les objets de base sont

- vecteurs, matrices
- data.frames, listes

Quelques fonctions génériques :

- `ls()` retourne la liste des objets de la session.
- `rm(a)` supprime l'objet `a`

Fonctions is/as

- `is.xxx(obj)` teste si `obj` est un objet de type `xxx`
- `as.xxx(obj)` contraint si possible `obj` au type d'objet `xxx` où `xxx` représente un type d'objet (complex, double, vector matrix etc...)

```
x=3
 is.double(x)
[1] TRUE
 is.complex(x)
[1] FALSE
```

```
 as.complex(x)
[1] 3+0i
 as.character(x)
[1] "3"
```

Remarque :

Conversion de TRUE / FALSE en valeur numérique :

```
 as.integer(T)
[1] 1
 as.integer(F)
[1] 0
```


Créer des vecteurs

- la fonction `c( )` concatène des scalaires ou des vecteurs :

```
 x=c(1,4,9)
 y=c(x,2,3)
 y
[1] 1 4 9 2 3
```

- Suites arithmétiques de raison 1 ou -1 : `c(a:b)`.

```
 c(1:4)
# a<b  raison 1
[1] 1 2 3 4

c(4:1)
 # a>b  raison -1
[1] 4 3 2 1

 c(1.4:7)
# a-b n'est pas un entier
[1] 1.4 2.4 3.4 4.4 5.4 6.4
```


- Généralisation : `seq(a,b,t)` où `a` est premier terme, le dernier $\leq b$ et la raison `t`

```
seq(from, to)                 la raison est 1
seq(from, to, by= )           on fixe la raison
seq(from, to, length.out= )   on fixe le nb de termes
```

par exemple

```
 seq(1,4,by =0.1)
 [1] 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7 ....
[26] 3.5 3.6 3.7 3.8 3.9 4.0
```

- `x=rep(y ,n)` pour créer un vecteur constitué de l'élément `y` répété `n` fois. (y peut être un scalaire ou un vecteur) par exemple

```
 rep(1,4)
 [1] 1 1 1 1
```


Créer des matrices

Les matrices sont créées avec la fonction `matrix`() à partir d'un vecteur. On doit fixer le nombre de colonnes `ncol` et/ou le nombre de lignes `nrow`.

```
 x = matrix(c(2,3,5,7,11,13),ncol=2)
x
     [,1] [,2]
[1,]    2    7
[2,]    3   11
[3,]    5   13
```

Par défaut la matrice est remplie colonne par colonne. Pour remplir ligne par ligne, on ajoute l'argument `byrow=T`

```
 y = matrix(c(2,3,5,7,11,13),ncol=2, byrow=T)
 y
         [,1] [,2]
[1,]    2     3
[2,]    5     7
[3,]    11   13
```

Attention : si la dimension du vecteur n'est pas égale au produit (`ncol` × `nrow`) alors l'opération effectuée est la suivante :

```
matrix(c(1:3), ncol=2,nrow=3)
     [,1] [,2]
[1,]    1    1
[2,]    2    2
[3,]    3    3

matrix(c(1:3), ncol=2)
     [,1] [,2]
[1,]    1    3
[2,]    2    1
```


Quelques matrices particulières : diagonale, Toeplitz

```
diag(1:4)
     [,1] [,2] [,3] [,4]
[1,]    1    0    0    0
[2,]    0    2    0    0
[3,]    0    0    3    0
[4,]    0    0    0    4
```

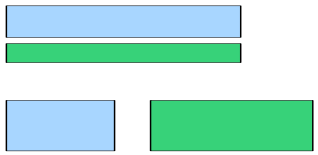
```
toeplitz(1:4)
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    2    1    2    3
[3,]    3    2    1    2
[4,]    4    3    2    1
```

diag

La fonction `diag` retourne une matrice diagonale lorsque le paramètre d'entrée est un vecteur.
Si le paramètre d'entrée est une matrice, alors elle retourne un vecteur constitué de la diagonale de la matrice

Concaténer des vecteurs/matrices

- `rbind`
- `cbind`



```
x=1:10
y=x^2
rbind(x,y)
  [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
x    1    2    3    4    5    6    7    8    9    10
y    1    4    9   16   25   36   49   64   81   100
cbind(x,y)
      x   y
 [1,] 1   1
 [2,] 2   4
 [3,] 3   9
 [4,] 4  16
 [5,] 5  25
 [6,] 6  36
 etc
```


Extraire des éléments d'un vecteur ou d'une matrice

```
vect=c(1.5:9.5 )
vect
[1] 1.5 2.5 3.5 4.5 5.5
6.5 7.5 8.5 9.5
```

```
mat=matrix(vect,ncol=3,nrow=3)
     [,1] [,2] [,3]
[1,]  1.5  4.5  7.5
[2,]  2.5  5.5  8.5
[3,]  3.5  6.5  9.5
```

Extraire un élément

```
vect[1]
[1] 1.5
mat[2,1]
 [1] 2.5
```

Colonne/ligne d'une matrice

```
mat[,1]
[1] 1.5 2.5 3.5
mat[3,]
    [1] 3.5 6.5 9.5
```

Extraire un bloc ou plusieurs coordonnées

```
mat[2:3,1:2]
     [,1] [,2]
[1,]  2.5  5.5
[2,]  3.5  6.5

vect[c(1,3,7)]
 [1] 1.5 3.5 7.5
```

Attention : `vect[-j]` retourne le vecteur `vect` sans la j ème coordonnée

```
vect[-c(1,3,7)]
2.5 4.5 5.5 6.5 8.5 9.5
```

Opérations sur les Matrices/Vecteurs

• Les opérations + * - / entre 2 vecteurs ou matrices de même dimension sont des opérations terme à terme.

```
 x
[1] 1 2 3 4 5
 y
[1] 0 0 0 1 1

 x*y
[1] 0 0 0 4 5
```

```
     A
     [,1] [,2]
[1,]    2    1
[2,]    4    9
 B
     [,1] [,2]
[1,]    0    2
[2,]    1    1
 A*B
     [,1] [,2]
[1,]    0    2
[2,]    4    9
```


Attention

Si les vecteurs ne sont pas de même longueur, le plus court est complété automatiquement .

```
 x =c(1:5)
 x
[1] 1 2 3 4 5
 y =c(1,2)
 y
[1] 1 2
 x + y
 [1] 2 4 4 6 6
```

```
      x  : 1 2 3 4 5
 y  : 1 2 1 2 1
-----------------
      x+y : 2 4 4 6 6
```


Quelques opérations particulières sur les matrices

```
 a=matrix(1,ncol=2,nrow=2)
 a
     [,1] [,2]
[1,]    1    1
[2,]    1    1
 a+3            #matrice  +scalaire
     [,1] [,2]
[1,]    4    4
[2,]    4    4
 a+c(1:2)       #matrice + vecteur
     [,1] [,2]
[1,]    2    2
[2,]    3    3
```


Action d'une fonction sur un vecteur ou une matrice

Soit FUN une fonction définie sur les scalaires qui retourne un scalaire. Par exemple

```
sqrt                square root
abs                 absolute value
sin   cos   tan     trigonometric functions (radians)
exp   log           exponential and natural logarithm
log10               common logarithm
gamma lgamma        gamma function and its natural log
```

Lorsque le paramètre d'entrée est un vecteur (respectivement une matrice), la fonction FUN est appliquée sur chacune des composantes. L'objet retourné est un vecteur (respectivement une matrice).

Exemple

Si $A = (a_{i,j})$ est une matrice, alors `exp(A)` retourne une matrice constituée des éléments $e^{a_{i,j}}$.

Quelques fonctions sur les matrices

- Le produit matriciel est obtenu avec `%*%`
- Calcul des valeurs/vecteurs propres :`eigen`
- Calcul du déterminant : `det`
- `t(A)` retourne la transposée de la matrice A
- décomposition de Choleski :`chol (X)` retourne R telle que $X = R'R$ où R est une matrice triangulaire supérieure et R' est la transposée de R.
- décomposition svd $X = UDV'$: renvoie une liste dont la composante d est le vecteur des valeurs singulières et non la matrice D

solve

- `solve(A)` retourne l'inverse de la matrice A
- `solve(A,b)` retourne x tel que $Ax = b$

Objets booléens et instructions logiques

- Les opérations logiques : `< , > , <= , >= , != [différent], == [égal]` retournent `TRUE` ou `FALSE`.
- La comparaison entre deux vecteurs est une comparaison terme à terme.
- Si les vecteurs ne sont pas de même longueur, le plus court est complété automatiquement.

```
a= 1:5   ;   b=2.5
a<b
[1]  TRUE  TRUE FALSE FALSE FALSE
```

Il est possible de définir plusieurs conditions à remplir avec les opérateurs

- ET : &
- OU : |

Pour extraire les éléments d'un vecteur `vect`, on peut utiliser des instructions logiques.

- Soit I un vecteur de booléens de même longueur que `vect` : `vect[I]` retourne les coordonnées `vect[j]` telles que $I[j] = TRUE$.

Applications

- extraire les composantes >8
  `vect[vect>8]` : `vect>8` est un vecteur de `TRUE` et `FALSE`, on extrait les composantes affectées à `TRUE`.
- extraire les composantes >8 ou <2
  `vect[(vect>8) | (vect<2)]`
- extraire les composantes >8 et <10
  `vect[(vect>8) & (vect<10)]`

Les valeurs manquantes : `NA`

`NA` (*not available*) code une valeur manquante. Elle se **propage** : toute opération impliquant `NA` vaut `NA`.

```
x <- c(1, 2, NA, 4)
sum(x)                  # NA
mean(x)                 # NA
sum(x, na.rm = TRUE)    # 7
mean(x, na.rm = TRUE)   # 2.333333
```

Attention, on ne teste pas l'égalité à `NA` :

```
x == NA                 # NA NA NA NA  (et non TRUE/FALSE)
is.na(x)                # FALSE FALSE TRUE FALSE
which(is.na(x))         # 3
sum(is.na(x))           # compter les manquantes
```

NA, NULL, NaN, Inf

Quatre objets distincts, souvent confondus.

Objet	Signification	Exemple
NA	donnée manquante	valeur non relevée
NULL	objet vide, de longueur 0	length(NULL) vaut 0
NaN	résultat indéfini	0/0
Inf	infini	1/0, -1/0

```
c(1, NA, 3)      # longueur 3 : le trou occupe une place
c(1, NULL, 3)    # longueur 2 : NULL disparait

is.na(NaN)       # TRUE  : NaN est un cas particulier de NA
is.nan(NA)       # FALSE : la reciproque est fausse
is.finite(1/0)   # FALSE
```

Où cela se manifeste : `summary()` et `str()` signalent les NA, et beaucoup de fonctions (`mean`, `sd`, `cor`, `lm`) ont un argument pour les traiter.

Effet de la précision sur la comparaison de réels

Est ce que $\sqrt{2}^2 = 2$?

```
 (sqrt(2)^2 == 2)
[1] FALSE
```

Une solution

```
 all.equal(sqrt(2)^2,2)
[1] TRUE
#ou
isTRUE(all.equal(sqrt(2)^2,2))
```


Fonction which

Soit vec un vecteur logique. La fonction which(vec) retourne les indices des coordonnées du vecteur vec qui prennent la valeur TRUE

```
 x=(1:10)^2
 x
 [1]   1   4   9  16  25  36  49  64  81 100
 which(x== 25)
[1] 5
 which(x > 21)
[1]  5  6  7  8  9 10
```

Exemple

Les commandes x[x>1] et x[which(x>1)] retournent le même vecteur.

Cas particulier which.max(x)

- which.max(x) retourne le premier indice réalisant le maximum.
- Attention c'est différent de which(x == max(x)) qui renvoie tous les indices réalisant le maximum.
- which.max(x) retourne which(x == max(x))[1]

Ces fonctions retournent un scalaire :

- sum() (somme $\sum_i x_i$), prod() (produit $\prod_i x_i$), mean() (moyenne $\frac{1}{n}\sum_{i=1}^n x_i$)
- max(), min()
- length() (longueur du vecteur),
- dim(), ncol(), nrow() (dimension de la matrice/nombre de lignes / nombre de colonnes.)

Ces fonctions retournent un vecteur :

- cumsum() (sommes cumulées $(x_1, x_1 + x_2, \ldots, \sum_{i=1}^n x_i)$, cumprod() (produits cumulés),
- sort (tri), order, unique
  remarque : sort(x) = x[order(x)]
- fft() (transformée de Fourier)

Les `data.frame`

Un `data.frame` est une structure de données tabulaire.

- chaque **ligne** représente une observation ;
- chaque **colonne** représente une variable ;
- les colonnes peuvent être de types différents :
  - numérique,
  - logique (`TRUE`/`FALSE`),
  - caractère,
  - facteur (`factor`), etc.

```
data1 <- data.frame(
  x1 = 1,
  x2 = 1:10,
  a  = letters[1:10]
)

data1
```

Par défaut, les lignes sont numérotées de 1 à n.

Manipuler un `data.frame`

Les fonctions les plus utiles sont

- `head()` : afficher les premières lignes ;
- `names()` : afficher ou modifier le nom des colonnes ;
- `row.names()` : afficher ou modifier le nom des lignes ;
- `dim()` : obtenir le nombre de lignes et de colonnes.

```
names(data1)
names(data1) <- c("c1","c2","c3")

head(data1,3)

dim(data1)

row.names(data1) <- letters[1:10]
head(data1,2)
```


Opérations arithmétiques

Si deux `data.frame` ont les mêmes dimensions et que leurs colonnes sont numériques, les opérations sont effectuées **élément par élément**.

```
A <- data.frame(x=1:3, y=2:4)
B <- data.frame(xx=1, yy=1:3)

A+B
```

$$\begin{matrix} 1+1 & 2+1 \\ 2+1 & 3+2 \\ 3+1 & 4+3 \end{matrix}$$

Extraire des données

La syntaxe est identique à celle des matrices.

Extraire une colonne

```
A$x
A[,1]
```

Extraire une ligne

```
A[2,]
```

Extraire un bloc

```
A[1:2,]
A[,1:2]
```

Assembler des `data.frame`

Deux fonctions sont très utilisées.

Ajouter des colonnes

```
cbind(A,B)

# ou

data.frame(A,B)
```

Ajouter des lignes

```
rbind(A,A)
```

Les colonnes doivent alors porter les mêmes noms.

Le type `factor`

Un `factor` code une variable **qualitative** : un ensemble fini de modalités, appelées *niveaux* (`levels`).

```
groupe <- factor(c("temoin", "traite", "temoin", "traite"))
levels(groupe)     # "temoin" "traite"  (ordre alphabetique par
    defaut)
table(groupe)      # effectifs par modalite
nlevels(groupe)    # 2
```

Un facteur est stocké comme un **vecteur d'entiers** plus une table de correspondance, d'où le piège suivant :

```
f <- factor(c(10, 20, 30))
as.numeric(f)                 # 1 2 3   <-- les INDICES des
    niveaux
as.numeric(as.character(f))   # 10 20 30  <-- la bonne conversion
```

À retenir : pour récupérer les valeurs d'un facteur numérique, toujours passer par `as.character`.

`factor` : ordre des niveaux

L'ordre des niveaux est alphabétique par défaut, ce qui est rarement l'ordre souhaité. Il gouverne pourtant l'ordre des barres d'un graphique, des colonnes d'un `table`, et la modalité de référence d'un modèle.

```
taille <- factor(c("petit", "grand", "moyen"))
levels(taille)      # "grand" "moyen" "petit"  : alphabetique

niv <- c("petit", "moyen", "grand")
taille <- factor(taille, levels = niv)
levels(taille)      # l'ordre voulu
```

Pour une variable réellement ordonnée, on peut le déclarer :

```
taille <- factor(taille, levels = niv, ordered = TRUE)
taille[1] < taille[2]     # la comparaison a un sens
```


Explorer un jeu de données

- `str()` : structure du tableau.
- `summary()` : résumé statistique.
- `head()` : premières lignes.
- `tail()` : dernières lignes.

```
str(data1)
summary(data1)
head(data1)
tail(data1)
```

Les listes I

Une **liste** est un objet pouvant contenir plusieurs objets, éventuellement de types différents.

Chaque élément peut être :

- un scalaire ;
- un vecteur ;
- une matrice ;
- un `data.frame` ;
- une autre liste ;
- une fonction ou tout autre objet R.

Les listes II

Les listes sont créées avec la fonction `list()`.

Exemple

On construit une liste `rnd` contenant :

- `serie` : un vecteur de 100 valeurs ;
- `taille` : un entier ;
- `type` : une chaîne de caractères.

```
rnd <- list(
   serie = 1:100,
   taille = 100,
   type = "arithm"
)
```


Nommer les éléments d'une liste

Les éléments d'une liste peuvent être nommés ou non.

Liste avec noms

```
rnd <- list(
   serie=1:100,
   taille=100,
   type="arithm"
)
```

Liste sans noms

```
rnd <- list(
   1:100,
   100,
   "arithm"
)
```

Conseil

Il est préférable de donner un nom aux éléments d'une liste. Le code est plus lisible et plus facile à maintenir.

Explorer une liste

Quelques fonctions utiles.

- `names()` : nom des composantes ;
- `length()` : nombre d'éléments ;
- `str()` : structure détaillée ;
- `summary()` : résumé (lorsqu'il existe).

```
names(rnd)

length(rnd)

str(rnd)

summary(rnd)
```

Accéder aux éléments d'une liste

Plusieurs syntaxes sont possibles.

Par son nom

```
rnd$taille
```

Par son indice

```
rnd[[2]]
```

Extraire une sous-liste

```
rnd[2]
```

Attention

- `rnd[[2]]` renvoie l'objet contenu dans la liste.
- `rnd[2]` renvoie une liste contenant un seul élément.

Modifier une liste

On peut ajouter, modifier ou supprimer des éléments.

```
rnd$auteur <- "Anne"

rnd$taille <- 120

rnd$type <- NULL
```

On peut également ajouter plusieurs éléments avec

```
rnd <- c(
   rnd,
   list(date="2026")
)
```


Pourquoi les listes ?

De nombreuses fonctions de R renvoient une liste.
Par exemple :

```
mod <- lm(dist ~ speed, data=cars)

names(mod)
```

Le résultat contient notamment :

- les coefficients ;
- les résidus ;
- les valeurs ajustées ;
- les données utilisées ;
- de nombreuses informations complémentaires.

Les listes constituent donc une structure fondamentale de R.



SECTION 3

Exercices

Énoncé — Matrices

1. Créer une matrice `M` 3x3 contenant les entiers de 1 à 9, remplie **par colonne**.
2. Créer une matrice `N` de même dimension, remplie **par ligne**.
3. Calculer `M + N`, `M %*% N` (produit matriciel) et `M * N` (produit terme à terme). Comparer.
4. Extraire la 2 ème ligne de `M`, puis la 3ème colonne de `N`.
5. Calculer la transposée de `M`. Vérifier que `t(t(M))` redonne `M`
6. Calculer le produit matriciel de `diag(1:3)` avec `M`.

Énoncé — Data frames

On utilise le jeu de données `iris` (`data(iris)`).

1. Afficher les 5 premières lignes (`head`) et la structure (`str`).
2. Quelles sont les dimensions du data.frame ?
3. Extraire la colonne `Sepal.Length`, de deux façons différentes.
4. Créer un sous-ensemble `iris_setosa` ne gardant que les lignes où `Species == "setosa"`.
5. Ajouter une colonne `Sepal.Ratio`, égale à `Sepal.Length / Sepal.Width`.
6. Comparer la moyenne de `Petal.Length` sur tout le tableau et sur `iris_setosa`.

Énoncé — Listes

1. Créer une liste `etudiant` contenant : `nom` (chaîne), `notes` (vecteur de 5 notes), `admis` (booléen).
2. Accéder à `notes` par son nom (`$`), puis par son indice (`[[ ]]`).
3. Calculer la moyenne des notes et l'ajouter à la liste sous le nom `moyenne`.
4. Afficher la structure complète de la liste (`str`).
5. Créer une liste `promo` contenant 3 listes `etudiant`. Extraire la moyenne du 2ème étudiant.



SECTION 4

Importer et exporter des données

Où suis-je ? Le répertoire de travail

R cherche et écrit les fichiers dans un répertoire courant, qui n'est pas forcément celui auquel on pense.

```
getwd()                          # repertoire courant
list.files()                     # ce que R voit depuis ce repertoire
setwd("/chemin/vers/mon/TP")     # le changer
```

L'erreur la plus fréquente en TP : `cannot open file 'donnees.csv': No such file or directory`. Dans neuf cas sur dix le fichier existe, mais pas là où R regarde.

Pourquoi soigner l'importation ?

L'importation est la premiere etape d'une analyse.

Il faut verifier :

- le separateur de colonnes ;
- le separateur decimal ;
- l'encodage des caracteres ;
- les noms de variables ;
- les valeurs manquantes ;
- les types de colonnes.

Importer un fichier CSV

Avec le package readr :

```
donnees <- read_csv("data/notes.csv")
```

Pour un fichier francophone avec point-virgule :

```
donnees <- read_csv2("data/notes.csv")
```

`read_csv2()` suppose souvent :

- des colonnes separees par `;` ;
- des nombres decimaux avec une virgule.

Importer un fichier texte general

Pour controler explicitement les options :

```
donnees <- read_delim(
  file = "data/tableau.txt",
  delim = ";"
)
```

Options utiles :

- `col_names = TRUE` : premiere ligne = noms des colonnes ;
- `na = c("", "NA", "NR")` : valeurs manquantes ;
- `locale()` : encodage et formats numeriques.

Importer un fichier Excel

Le package `readxl` permet de lire des fichiers `.xls` et `.xlsx`.

```
library(readxl)

donnees <- read_excel("data/enquete.xlsx")
```

Choisir une feuille :

```
donnees <- read_excel(
  "data/enquete.xlsx",
  sheet = "reponses"
)
```


Verifier les donnees importees

Quelques reflexes apres l'importation :

```
glimpse(donnees)
head(donnees)
summary(donnees)
```

Questions a se poser :

- Les colonnes ont-elles le bon type ?
- Les noms de variables sont-ils clairs ?
- Les valeurs manquantes sont-elles bien reconnues ?
- Les nombres sont-ils lus comme des nombres ?

Exporter un tableau

Exporter en CSV :

```
write_csv(donnees, "resultats/donnees_nettoyees.csv")
```

Exporter en format francophone :

```
write_csv2(donnees, "resultats/donnees_nettoyees.csv")
```

Bonne pratique : separer les dossiers `data/`, `scripts/` et `resultats/`.

Sauvegarder des objets R

Pour sauvegarder un objet R :

```
saveRDS(donnees, "resultats/donnees.rds")
```

Pour le relire :

```
donnees <- readRDS("resultats/donnees.rds")
```

Le format `.rds` conserve les types R : facteurs, dates, listes, etc.

Exemple

1. Sauvegarder des objets R dans un fichier

```
x=1:10
y=list(a=1 , b=TRUE ,c="exemple")
save(x,y,file='sav.rda')
```

2. Lire un fichier qui contient des objets R

```
load("sav.rda")
x
[1]  1  2  3  4  5  6  7  8  9 10
```

   Attention si un objet R appelé `x` (ou `y`) existait avant l'appel de la fonction `load`, il a été remplacé par celui contenu dans le fichier `sav.rda`

3. `saveRDS` peut aussi être utiliser si on sauvegarde une unique liste. Le fichier est lu avec `readRDS`. On peut changer le nom de la liste à la lecture du fichier

```
a= list(x=1,y=3)
saveRDS(a,'sav.rds')
```

```
b = readRDS('sav.rds')
b
$x
[1] 1
$y
```


Importer et exporter des fichiers texte

1. **Importer un vecteur**

```
x <- scan("data.dat")
```

   Cette commande lit une suite de valeurs numériques stockées dans le fichier `data.dat` et les place dans le vecteur `x`.

2. **Importer un tableau de données**

```
x <- read.table("data.dat")
x <- read.table("data.dat", header = TRUE)
```

   L'option `header = TRUE` indique que la première ligne du fichier contient les noms des colonnes.

3. **Exporter des données**

```
write(x, "data.dat")
write.table(x, "data.dat")
```

   - `write()` est adapté aux vecteurs.
   - `write.table()` permet d'exporter un tableau de données (`data.frame`).

SECTION 5

Autour des lois de probabilités

Généralités

Soit X une variable aléatoire de loi P_X

$$P_X(A) = P(X \in A) = \begin{cases} \sum_{x \in A} P(X = x) & \text{loi discrète} \\ \int_A f(x)\mathrm{d}x & \text{loi continue} \end{cases}$$

Pour les lois classiques, des fonctions existent pour

- calculer
  - la densité $\begin{cases} P(X = x) & \text{pour les lois discrètes} \\ f(x) & \text{pour les lois continues} \end{cases}$
  - la fonction de répartition $F(x) = P(X \leq x)$
  - les quantiles $F^-(u) = \inf\{x : \ F(x) \geq u\}$ [1]
- simuler des nombres aléatoires suivant la même loi que X.

[1] Si F est une bijection alors $F^- = F^{-1}$

Quelques lois disponibles

1. Lois discrètes
   - Loi binomiale (n,p) — `binom`
   - Loi hypergéométrique (N,n,k) — `hyper`
   - Loi de Poisson (a) — `pois`
   - Loi géométrique (p) — `geom`
   - Loi à support fini $\{(a_i, p_i), i = 1...m\}$ — `sample`
2. Lois continues
   - Loi Gaussienne (m,σ^2) — `norm`
   - Loi uniforme sur $[a, b]$ — `unif`
   - Loi de Student à ν degrés de liberté — `t`
   - Loi du χ^2 à ν degrés de liberté — `chisq`

Attention à la paramétrisation des lois I

Loi	nom	paramètres		
beta	beta	shape1	shape2	ncp
binomial	binom	size	prob	
Cauchy	cauchy	location	scale	
chi-squared	chisq	df	ncp	
exponential	exp	rate		
F	f	df1	df2	ncp
gamma	gamma	shape	scale	
geometric	geom	prob		
hypergeometric	hyper	m	n	k
log-normal	lnorm	meanlog	sdlog	
logistic	logis	location	scale	
negative binomial	nbinom	size	prob	
normal	norm	mean	sd	
Poisson	pois	lambda		
Student	t	df	ncp	
uniform	unif	min	max	
Weibull	weibull	shape	scale	
Wilcoxon	wilcox	m	n	

Si www représente le nom d'une des lois alors

- `d`www(x,...) calcule la densité de la loi www au point x
- `p`www(x,...) : calcule la fonction de répartition au point x
- `q`www(α,...) : calcule le quantile d'ordre α
- `r`www(n,...) retourne un échantillon de taille n

les ... représentent les paramètres spécifiques à chaque loi.

Exemple : loi gaussienne de moyenne 0 et de variance 1

- `d`norm(x,0,1) : densité au point x
- `p`norm(x,0,1) : fonction de répartition au point x
- `q`norm(α,0,1) : quantile d'ordre α
- `r`norm(n,0,1) : échantillon de taille n

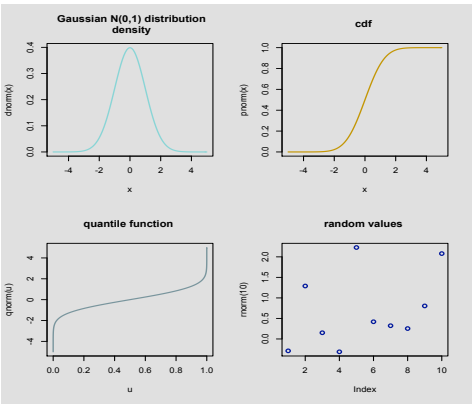
```
x=seq(-5,5,.01)
u=seq(0,1,.01)

plot(x,dnorm(x,0,1),
     type="l")

plot(x,pnorm(x,0,1) ,
     type="l")

plot(u,qnorm(u,0,1) ,
     type="l")

plot(rnorm(10,0,1))
```



Reproductibilité : la fonction `set.seed`

Les générateurs de R sont *pseudo*-aléatoires : la suite produite est entièrement déterminée par un état interne, la *graine*. Fixer la graine rend la suite reproductible.

```
set.seed(42)
rnorm(3)          # trois valeurs

rnorm(3)          # trois AUTRES valeurs : la graine a avance

set.seed(42)
rnorm(3)          # a nouveau les trois premieres
```

Trois raisons de le faire systématiquement :

- vos résultats sont reproductibles par un collègue, un correcteur, vous-même dans six mois ;
- un bug qui n'apparaît qu'une fois sur cent devient reproductible ;
- on compare deux méthodes *sur les mêmes tirages*, donc sans bruit parasite.

`set.seed` : quelques précautions

```
set.seed(1)
x <- rnorm(10)
y <- rnorm(10)    # y n'est PAS egal a x : la graine a avance

set.seed(1) ; x <- rnorm(10)
set.seed(1) ; y <- rnorm(10)    # cette fois y est identique a x
```

- La graine agit sur *tout* le flux aléatoire : appeler `sample` entre deux `rnorm` change la suite obtenue.
- Il faut donc placer `set.seed` **en tête de script**, une seule fois, et non avant chaque tirage.
- La valeur de la graine n'a aucune importance ; ce qui compte est qu'elle soit fixée et écrite dans le script.
- Pour un TP rendu : mettre `set.seed` en première ligne garantit que les chiffres du rapport correspondent au code.

Simulation d'une loi discrète finie

Soit Z une variable aléatoire à valeurs dans $\{x_1, ..., x_k\}$ telle que

$$p(Z = x_i) = prob_i$$

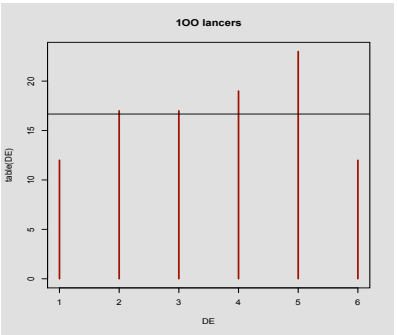
pour tout $i = 1...k$

- La fonction `sample` est un générateur de nombres aléatoires pour les lois discrètes à support fini.
- La syntaxe est `sample(x, n, replace= TRUE, prob)` où
  - x contient les valeurs prises par Z
  - *prob* contient les valeurs des probabilités. lorsque l'option `prob` n'est pas spécifiée, par défaut c'est la loi uniforme Lorsque `replace=TRUE` indique que le tirage est avec remise. Autrement dit on simule une réalisation de n variables aléatoires indépendantes et de même loi

Simuler le résultat d'un dé à 6 faces

On simule suivant la loi uniforme sur l'ensemble $\{1, ..., 6\}$.
On simule $n = 100$ réalisations de façon indépendante

```
# Les réalisations
 DE
  [1] 3 2 5 3 4 5 4 4 2 4 2 5 2
      6 5 6 4 3 5 4 5 3 5
 [24] 3 5 4 4 5 5 4 4 4 3 5 5 2
      5 3 5 4 3 1 3 1 3 4
 [47] 2 5 6 2 1 5 1 6 2 1 2 3 5
      1 3 4 2 3 1 2 6 1 6
 [70] 6 4 1 3 4 5 6 1 6 3 5 4 1
      5 2 2 2 3 3 2 2 6 6
 [93] 6 4 2 5 5 1 5 4
# calcul du nombre de 1,...,6
     dans
#l'échantillon simulé :
  table(DE)
DE
 1  2  3  4  5  6
12 17 17 19 23 12
# représentation de la ré
     partition
 plot(table(DE))
```



Construire une fonction càdlàg constante par morceaux

La fonction `stepfun` retourne un objet de type `function` qui définit une fonction constante par morceaux et continue à droite.
La syntaxe est
`F=stepfun(x, z)`
où

- le vecteur `x` contient les points de discontinuité de la fonction,
- le vecteur `z` est de la forme `c(a, y)` où `a` est la valeur prise par F avant le premier point de discontinuité et y les valeurs de la fonction aux points de discontinuités `x`.

Pour représenter graphiquement la fonction F on utilise `plot` ou `lines` (pour superposer) : avec l'argument `vertical=FALSE`

```
plot(F, vertical=FALSE)  ou lines(F, vertical=FALSE)
```

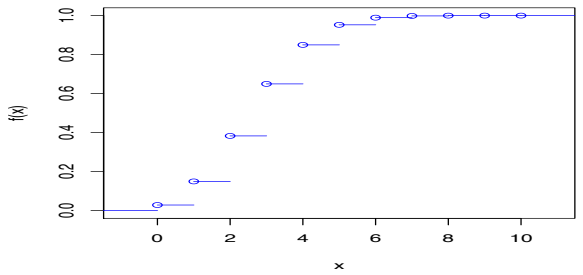
Application : les fonctions de répartition des lois discrètes

Exemple : loi binomiale

On trace la fonction de répartition de la loi binomiale $B(10, .3)$

```
points.discont = 0:10
F.points.discont = pbinom(points.discont,10,0.3)

F = stepfun( points.discont , c(0,F.points.discont) )
plot(F,vertical =FALSE ,col=4,main="fct rep loi binomiale" ,ylab="F
    ")
```



SECTION 6

Exercices

Loi normale

On considère une taille $X \sim \mathcal{N}(170, \sigma^2 = 10)$ (en cm).

1. Calculer $P(X \leq 180)$.
2. Calculer $P(160 \leq X \leq 180)$.
3. Calculer le quantile d'ordre 0.95 de X.
4. Calculer la densité de X au point $x = 170$.
5. Simuler un échantillon de 1000 valeurs de X ; comparer moyenne et écart-type empiriques aux valeurs théoriques.

Énoncé — Loi de Poisson

On considère $X \sim \mathcal{P}(4)$.

1. Calculer $P(X = 0)$.
2. Calculer $P(X \leq 2)$.
3. Calculer $P(X \geq 6)$.
4. Calculer $P(3 \leq X \leq 8)$ de deux façons : par une somme de `dpois`, puis par différence de `ppois`.
5. Calculer le quantile d'ordre 0.99 de X.
6. Simuler 1000 réalisations de X ; comparer variance empirique et théorique (rappel : pour Poisson, variance = moyenne = λ).

SECTION 7

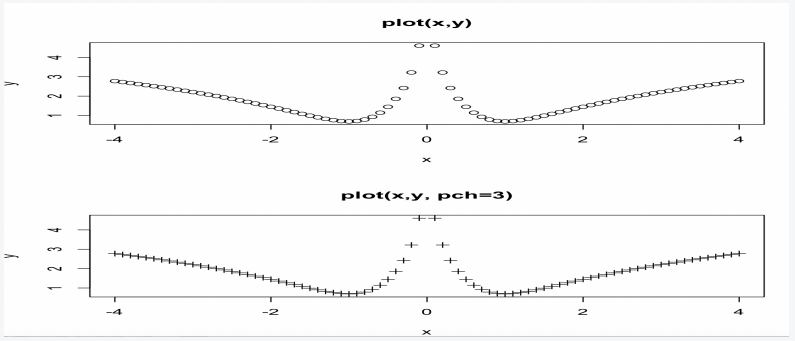
Les graphiques

Les fonctions usuelles `plot()`, `lines()`, `points()`

- `plot` est la fonction centrale
- Les fonctions `points` ou `lines` sont utilisées pour superposer des courbes ou des nuages de points.

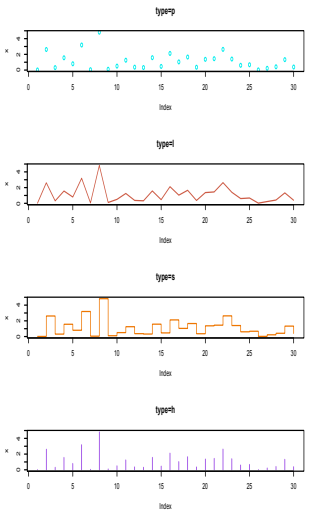
Premier exemple : représenter des vecteurs `plot(y)` / `plot(x,y)`

```
x= seq(-4,4,.1)          y=log(x^2+1/x^2)
```



Quelques arguments de la fonction `plot`

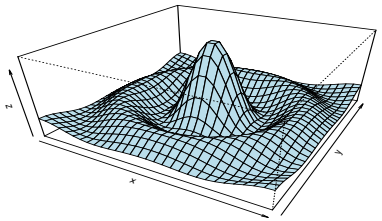
- pour fixer les limites des axes
  - `ylim=c(ay,by)` et `xlim=c(ax,bx)`
  - par défaut les bornes sont optimisées sur la première courbe tracée
- `type="p"` (points) ou `"l"` (ligne) : pour tracer une ligne ou un nuage de points.
- `pch` : type de points
- `lty` : type de lignes.
- `col` : couleur



Graphique en 3D

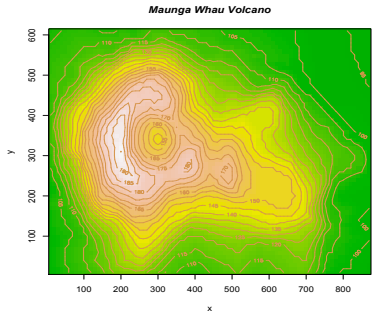
représentation d'une fonction de $\mathbb{R}^2 \to \mathbb{R}$

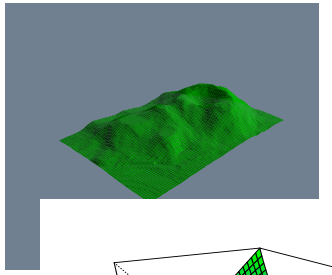
```
x <- seq(-10, 10, length= 30)
y <- x
f <- function(x,y) { r <- sqrt(x^2+y^2); 10 * sin(r)/r }
z <- outer(x, y, f)
persp(x, y, z, theta = 30, phi = 30, expand = 0.5, col = "
    lightblue")
```

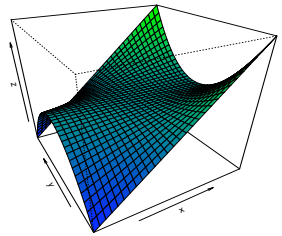


Représentation graphique d'une matrice

contour(x, y, matrice....) image( ) persp( )

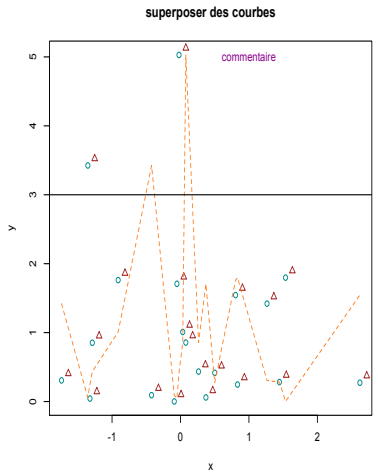






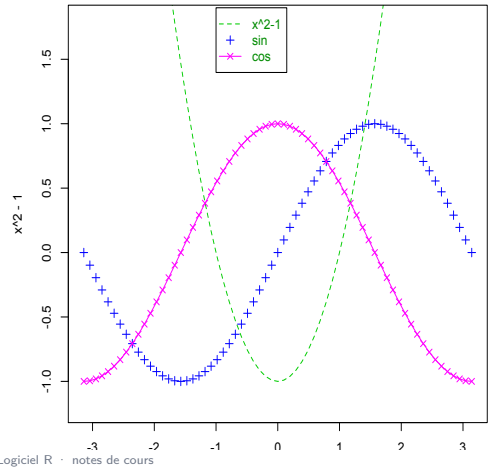
Superposition de courbes

```
 x=rnorm(20)          y=rexp(20)
#nuage de points
 plot(x,y)
   #ajouter un nuage de points
 points(x+.1,y+.1, pch=2)
  #ajouter une ligne
 lines(sort(x),y, lty=2)
  #ajouter une ligne
     horizontale
 abline(h=3)
 #texte + frametitle
 text(1,5,"commentaire")
title("superposer des courbes")
```

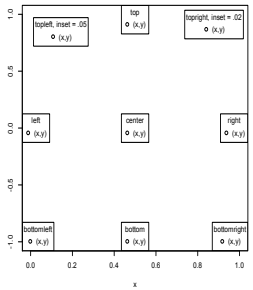


légende `legend`

```
legend(-1, 1.9, c("x^2-1","sin","cos"), col = c(3,4,6),
                                lty = c(2, -1, 1), pch = c(-1, 3, 4))
```



les emplacements prédéfinis :



Autour de la fonction `plot`

Le graphique produit par la fonction `plot(x)` dépend de la classe de l'objet `x`.

```
methods(plot)
 [1] "plot.data.frame"  "plot.default"
 [3] "plot.density"     "plot.factor"
 [5] "plot.formula"     "plot.function"
 [7] "plot.histogram"   "plot.lm"
 [9] "plot.mlm"         "plot.mts"
[11] "plot.new"         "plot.POSIXct"
[13] "plot.POSIXlt"     "plot.table"
[15] "plot.ts"          "plot.window"
[17] "plot.xy"
```


Illustrations

- sur une fonction (par ex `sin`)

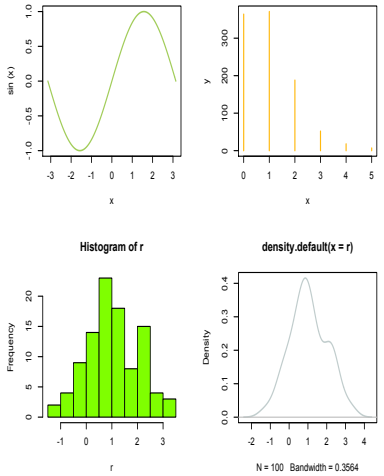
```
plot(sin,xlim=c(-pi,pi))
```

- sur un tableau

```
x=rpois(1000,1)
y=table(x)
y
x  0   1   2   3   4   6
  374 372 162  71  20   1
plot(y)
```

- sur un histogramme ou une densité

```
r=rnorm(100,1)
z=hist(r,plot=F)
plot(z)
w=density(r)
plot(w)
```



Représentation graphique d'une matrice ou dataframe

Les outils graphiques `matplot` et `pairs` sont adaptés aux matrices dont les colonnes correspondent à des variables.

Exemple

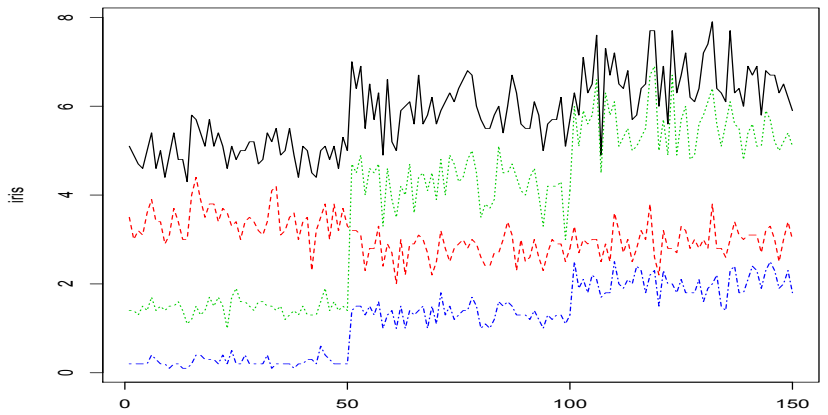
```
data(iris)
iris
```

	Sepal.Length	Sepal.Height	Petal.Length	Petal.Height	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4					
...					
54	5.5	2.3	4.0	1.3	versicolor
55	6.5	2.8	4.6	1.5	versicolor
...					
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica

Fonction `matplot(matrice ...)`

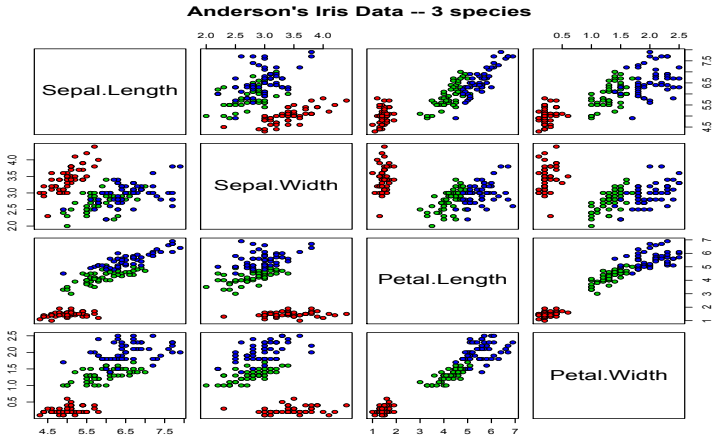
Cette fonction représente sur un même graphique les colonnes d'une matrice ou d'une data.frame.

Anderson's Iris Data – 3 species



Utilisation des biplots , `pairs(matrice , ....)`

Cette fonction représente tous les nuages de points possibles entre les différentes colonnes.



Construction d'un polygone

1. On commence par fixer les axes des abscisses et des ordonnés à l'aide d'un graphique vide.

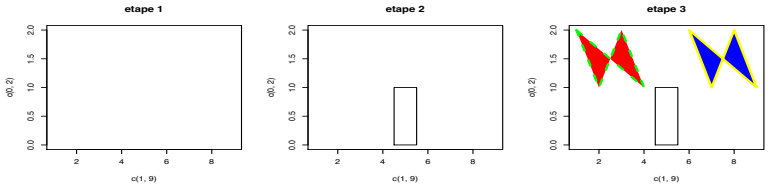
```
plot(c(1,9), c(0,2), type="n")
```

2. Avec la fonction `polygon`, on trace le polygone défini pas ses sommets

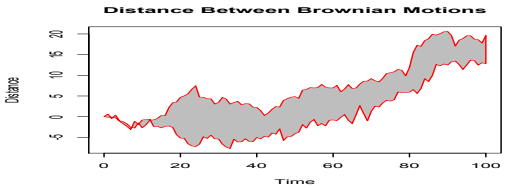
```
polygon(c(4.5,5.5,5.5,4.5) ,c(0,0,1,1))
```

3. Arguments supplémentaires

```
polygon(1:9, c(2,1,2,1,NA,2,1,2,1), col=c("red", "blue"),
     border=c("green", "yellow",  lwd=3, lty=c("dashed", "
    solid"))
```



autre exemple




```
n=100
z.haut=c(0,cumsum(rnorm(n)) )
z.bas= c(0,cumsum(rnorm(n)))

xx <- c(0:n, n:0)
yy <- c(z.bas, rev(z.haut))
#graphique vide pour fixer les dimensions
plot   (xx, yy, type="n", xlab="Time", ylab="Distance")
#tracer le polygone
polygon(xx, yy, col="gray", border = "red")
title("Distance Between Brownian Motions")
```


- Pour sauvegarder un graphique :
  - utilisation de la fonction `dev.print`.
    - pour obtenir un fichier postscript :
      `dev.print(postscript, file="essai.ps")`
    - pour obtenir un fichier pdf :
      `dev.print(pdf, file="essai.pdf")`
  - utilisation des menus (sous widows ou mac seulement)

- La fenêtre graphique peut être fractionnée en utilisant
  - `par(mfrow=c(n,m))`, on obtient alors $n \times m$ graphiques sur une même page organisés sur n lignes et m colonnes
  - `split.screen(m,n)`
    - `screen(i), screen(i,FALSE)` pour sélectionner la sous fenêtre
    - `erase.screen()`
    - `close.screen(all = TRUE)`

SECTION 8

Exercices

fonction

1. Tracer la représentation graphique les functions
   - $f_1(x) = x^2$ sur [-1,1]
   - $f_2(x) = x^3$ sur sur [-1,1]
   - $f_3(x) = \log(x)$ sur sur [1/2,1]

   Utiliser des couleurs différentes et mettre une legende

Énoncé — Théorème central limite

1. Pour $n \in \{1, 5, 30, 100\}$, simuler $N = 2000$ répétitions de la moyenne empirique de n variables i.i.d. $\mathcal{E}(1)$. Tracer les quatre histogrammes.
2. Superposer à chaque histogramme la densité de $\mathcal{N}(1, 1/\sqrt{n})$ (rappel : pour $\mathcal{E}(1)$, espérance et écart-type valent 1).
3. Commenter l'évolution de la forme de l'histogramme quand n augmente.
4. À partir de quelle taille d'échantillon n l'approximation normale vous semble-t-elle raisonnable, visuellement ?
5. Reprendre l'expérience avec une loi *très* asymétrique : une Bernoulli de paramètre $p = 0.05$ (événement rare) à la place de $\mathcal{E}(1)$. Le TCL est-il déjà satisfaisant pour $n = 30$? Que se passe-t-il pour $n = 500$?

SECTION 9

Définir une fonction dans R

Structure générale pour créer des fonctions

- La structure générale d'une fonction est

```
FUN=function(liste_des_paramètres)
{
  commandes
  return(objets_retournés)
  }
```

- Les accolades { et } définissent le début et la fin de la fonction.
- La dernière instruction `return` contient le ou les objets retournés par la fonction.
- Exécuter la fonction : `FUN(...)`

Exemple

La fonction suivante retourne le résultat de n lancers d'une pièce.

```
PF = function(n ,proba.pile)
{
#nb aléatoire suivant Unif(0,1)
u=runif(n)
pf=(u<proba.pile)
pf = as.integer(pf)
return(pf)
}
```

La sortie de la fonction est

```
PF(10,1/2)
 [1] 0 0 1 0 1 0 1 1 0 0
# avec affectation de la sortie dans le vecteur x
x = PF(10,1/2)
x
 [1] 0 1 1 1 0 0 0 0 0 0
```


Renvois multi-arguments

La fonction `return` interdit les sorties avec plusieurs arguments : il faut les regrouper dans un seul objet sous la forme d'une liste.

```
 PF = function(n ,proba.pile)
{
u=runif(n) #nb aléatoire suivant Unif(0,1)
pf=(u<proba.pile)
pf = as.integer(pf)
f =mean(pf)
return(list(echantillon = pf,frequence = f))
}
```

Exécution de la fonction :

```
 PF(4,1/2)
$echantillon
[1] 0 0 1 0
$frequence
[1] 0.25
```

```
 l= PF(4,1/2)
 l$echantillon
[1] 0 0 1 1
 l$frequence
[1] 0.5
```

Paramètres par défaut

On peut affecter des valeurs par défaut aux paramètres d'entrée d'une fonction.

Modification de la fonction PF

par défaut on suppose que la pièce est équilibrée.

```
PF = function(n ,proba.pile=1/2)
{
u=runif(n) #nb aléatoire suivant Unif(0,1)
pf=(u<proba.pile)
pf = as.integer(pf)
return(pf)
}
```

Les commandes

```
PF(10)    OU  PF(10,1/2)
```

retournent le même résultat

Paramètres d'entrée

Il y a trois façons de spécifier les paramètres d'entrée d'une fonction

- par la position: les paramètres d'entrée sont affectés aux premiers arguments de la fonction. PF(3,1/2) : les paramètres d'entrée sont n=3 et proba.pile=1/2
- par le nom: il s'agit du moyen le plus sûr, les noms des arguments sont précisés de manière explicite. On peut alors écrire PF(proba.pile=1/2 ,n=3), l'ordre n'est plus prioritaire
- avec des valeurs par défaut: ces valeurs par défaut seront utilisées si les paramètres d'entrée ne sont pas spécifiés. On peut alors écrire PF(3) ou PF(n=3) : les paramètres d'entrée sont n=3 et la valeur par défaut pour proba.pile c'est à dire 1/2.

Remarque sur les valeurs par défaut

Modification de la fonction PF:

on inverse l'ordre des paramètres d'entrée :

```
PF = function(proba.pile=1/2, n)
{
...
 }

 PF(10)
Erreur dans .Internal(runif(n, min, max)) : 'n' est manquant
 PF(n=10)
 [1] 0 0 1 0 1 0 1 0 1 0
```

IL est donc préférable de placer les paramètres sans valeur par défaut en premier dans la déclaration des variables d'entrée.

Pour les fonctions de deux variables

- f une fonction de deux variables $f : (x, y) \mapsto f(x, y)$
- x et y deux vecteurs de même dimension.

La commande `f(x,y)` retourne le vecteur constitué des éléments $f(x_i, y_i)$.

Si x et y ne sont pas de même dimension, celui de plus petite dimension est répété.

Tableaux croisés

La fonction `outer` retourne une matrice de la forme

$$M(i, j) = fun(x_i, y_j)$$

```
x=1:5
y=1:5
M=outer(x,y,'fun')
```

`fun` peut aussi être une opération élémentaire +/-*

la fonction Vectorize

Soit f une fonction dont le paramètre d'entrée x est un scalaire.
`Vectorize` transforme la fonction f en une fonction vectorielle c'est à dire une fonction qui évalue la fonction f en chaque point d'un vecteur d'entrée.
Soit $x = (x_1, ..., x_n)$, on veut évaluer f aux points x_i.

```
# on transforme la fonction {f} en une fonction vectorielle df.
 df = Vectorize (f, 'x')
y=df(x)
```

Autres programmations possibles

1. avec une boucle `for`

```
y = rep(0,n)
for ( i in 1:n) y[i] = f(x[i])
```

2. avec la fonction `sapply`

```
y = sapply(x, 'f')
```


Illustration sur des fonctions de 2 variables

```
 f=function(x,y) sin(x+y^2)
 f(1,1)
[1] 0.9092974
 #
 x = 1:3
 y=  1:3
 # on calcule f(x[i],y[i])
 f(x,y)
[1]  0.9092974 -0.2794155 -0.5365729
 z =1:2
 f(x,z)
[1]  0.9092974 -0.2794155 -0.7568025
Message d'avis :
In x + y^2 :
  la taille d'un objet plus long n'est pas multiple de la taille d'
    un objet plus court
 # le calcul effectué est
 #f(x[1],z[1]) f(x[2],z[2]) f(x[3],z[1])
 #identique à
 f(x,c(z,z[1]))
[1]  0.9092974 -0.2794155 -0.7568025
```


suite

```
 f(1,y)
[1]  0.9092974 -0.9589243 -0.5440211
 # identique à
 f(rep(3,1),y)
[1] -0.7568025  0.6569866 -0.5365729

 #calcul du tableau croisé f(x[i],y[j])
 df = Vectorize(f,'x')
 df(x,z)
          [,1]       [,2]       [,3]
[1,]  0.9092974  0.1411200 -0.7568025
[2,] -0.9589243 -0.2794155  0.6569866
 # les vecteurs  colonnes  sont f(1,1:2) f(2,1:2) f(3,1:2)
```


SECTION 10

Exercices

Énoncé — Fonctions

1. Écrire une fonction `f(x)` qui retourne $x^2 + 3 * sin(x)$.
2. Écrire une fonction `stats(x)` qui retourne une **liste** contenant le min, le max et la moyenne de `x`.
3. Modifier `stats` pour que la moyenne soit optionnelle, via un argument `moyenne = TRUE` par défaut.

SECTION 11

Structures de contrôle et Itérations

Instructions conditionnelles

La syntaxe
`if (condition) {instructions}` permet de calculer les instructions uniquement si la condition est vraie.
`if (condition) { A } else {B}` calcule les instructions `A` si la condition est vraie et les instructions `B` sinon.
Par exemple,
`if (x>0) y=x*log(x) else y=0`
Remarque : Si les instructions se limitent à un seul calcul comme dans cet exemple on peut utiliser la fonction `ifelse`
`y=ifelse(x>0,x*log(x),0)`

Opération non vectorielle

1. Avec la commande `if (condition) ... else ...` la condition ne peut pas être vectorielle
   Par exemple

```
x= 1:7
if(x>2) print("A") else print("B")
[1] "B"
```

   Dans la condition, x a une longueur supérieure à 1. Seul le premier élément est utilisé :

   (x>2) correspond à (x[1] >2)

2. `ifelse` permet d'appliquer une instruction conditionnelle sur chacune des coordonnées d'un vecteur.

```
ifelse(x<2,"A","B")
[1] "A" "B" "B" "B" "B" "B" "B"
```

   Alternative :

```
g = function(x) if(x>2) print("A") else print("B")
vg = Vectorize(g,'x')
vg(1:10)
```

Itérations

On utilise les boucles pour exécuter plusieurs fois une instruction ou un bloc d'instructions
Les trois types de boucle sont

- `for` :
  `for`(var `in` seq) {commandes}
- `while` :
  `while`(cond) { commandes}
- `repeat` :
  `repeat` { commandes ; `if` (cond) `break` }

1. Dans une boucle `for`, le nombre d'itérations est fixé.
2. La durée d'execution des boucles `while/repeat` peut être infinie !

Exemple

Les 3 programmations suivantes retournent le même résultat.

1. Avec l'instruction `for`

```
   for ( i in 1:10)
    {
commandes
 }
```

2. Avec l'instruction `while`.

```
i= 1
  while (i <= 10 )
   {
    commandes
     i = i+1
   }
```

3. Avec l'instruction `repeat`

```
i= 1
 repeat
   {
     commandes
     i = i+1
     if (i>10) break
  }
```


Exemple

On veut simuler n variables aléatoires suivant la loi de $X_1 + ... + X_p$ où les X_i sont iid suivant la loi uniforme sur $[0, 1]$.
On stocke l'échantillon dans Y.

```
# initialisation de Y avec des
    0
Y = rep(0,n)
for ( i in 1:n )
 {  Y[i] = sum(runif(p,0,1)) }
```

Une autre façon de remplir Y par concaténation :

```
#  Y est un  vecteur vide
Y = NULL
for ( i in 1:n )
 {
 Y  =  c(Y, sum(runif(p,0,1)))
}
```

Une autre programmation :

```
# initialisation de Y avec des
    0
Y = rep(0,n)
for( i in 1:p)
 {  Y = Y + runif(n,0,1) }
```


Exemple

On simule des variables aléatoires suivant la loi de Bernoulli $B(1/2)$ jusqu'à l'obtention du premier 1.
Le nombre de variables (noté N dans le code ci dessous) simulées suivant la loi de Bernoulli suit une loi géométrique de paramètre $1/2$.

En utilisant `while`

```
  x=rbinom(1,1,.5)
  N=1
while ( x != 1)
{
 x=rbinom(1,1,.5)
 N=N+1
}
```

En utilisant `repeat`

```

N=0
repeat {
 x=rbinom(1,1,.5)
N=N+1
if (x==1) break
}
```

Exemple

On dispose des températures moyennes mensuelles relevées à Nottingham pendant 20 ans de 1920 à 1939.

```
nottem
     Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct  Nov  Dec
1920 40.6 40.8 44.4 46.7 54.1 58.5 57.7 56.4 54.3 50.5 42.9 39.8
1921 44.2 39.8 45.1 47.0 54.1 58.7 66.3 59.9 57.0 54.2 39.7 42.8
    etc
nottem=matrix(nottem,ncol=12,byrow=T)
```

On souhaite calculer un profil moyen annuel et le stocker dans le vecteur `temp`.

```
temp = rep(0,12)
for (i in 1:12)  temp[i] = mean( nottem[,i] )
```

On peut aussi initialiser le vecteur `temp` comme le vecteur vide, puis on le remplit en concaténant les résultats

```
temp = NULL
for (i in 1:12)  temp = c(temp , mean( nottem[,i] ) )
```


La fonction `apply`

`apply(matrice , MARGIN, FUN, ARG.COMMUN)`

La fonction `apply()` permet d'appliquer la même fonction `FUN` sur toutes les lignes `(MARGIN=1)` ou les colonnes `(MARGIN=2)` d'une matrice `MAT`

- Chaque ligne (`(MARGIN=1)` ou les colonnes `(MARGIN=2)`) est affectée au premier paramètre de la fonction `FUN`
- La syntaxe de `FUN` est `FUN( x, ARG.COMMUN)`
  où
  - `x` est un vecteur
  - `ARG.COMMUN` représente éventuellement des paramètres supplémentaires qui sont communs à toutes les exécutions.

Cette fonction remplace une boucle sur le nombre de colonnes ou de lignes

Exemple d'utilisation de la fonction `apply`

Retour à l'exemple des températures

Calcul du profil annuel :

```
temp = apply(nottem,2,mean)              # moyenne sur les
    colonnes
[1] 39.695 39.190 42.195 46.290 52.560 ....
```

Calcul des moyennes annuelles

```
temp.annuelle = apply(nottem,1,mean) # moyenne sur les lignes
```


Supprimer une boucle en utilisant `apply`

Objectif : Estimer la distribution de la médiane empirique

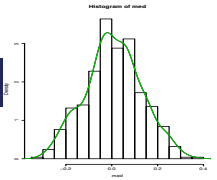
- On simule N échantillons de taille n iid suivant la loi gaussienne standard.
- Pour chaque échantillon, on calcule et on stocke la valeur de la médiane empirique dans le vecteur `med`.

1. avec boucle

```
n =100
N=500
med = 1:N
for (i in 1:N) med[i] = quantile(rnorm(n,0,1), probs=1/2)
```

2. Sans boucle

```
 Alea  = matrix(rnorm(n*N,0,1) , ncol= N)
med = apply(Alea,2,quantile, probs=1/2)
```



replicate

La fonction `replicate` évalue *n* fois une même expression

[utile l'expression implique des nombres aléatoires !!].

La syntaxe : `replicate(n, expression)`

Exemples

```
replicate(10,1+1)
 [1] 2 2 2 2 2 2 2 2 2 2

x=rnorm(10)
replicate(5,mean(x))
[1] -0.5073058 -0.5073058 -0.5073058 -0.5073058 -0.5073058

replicate(5,mean(rnorm(10)))
[1]  0.22137912  0.09330663 -0.12511439 -0.02613061 -0.19182371
```


Suite de l'exemple sur la médiane empirique

La fonction `replicate` permet de programmer sans boucle le code suivant

```
n =100
N=500
med = 1:N
for (i in 1:N) med[i] = quantile(rnorm(n,0,1), probs=1/2)
```

La syntaxe est la suivante

```
med = replicate(500 , quantile(rnorm(100,0,1),probs=1/2))
```


sapply,lapply

Ces fonctions calculent la même fonction sur tous les éléments d'un vecteur ou d'une liste.
La syntaxe :

`lapply(X,FUN, ARG.COMMUN)`

- La fonction `lapply` applique une fonction `FUN` à tous les éléments du vecteur ou de la liste `X`.
- Les valeurs de `X` sont affectées au premier argument de la fonction `FUN`.
- Si la fonction FUN a plusieurs paramètres d'entrée, ils sont spécifiés après le nom de la fonction : `ARG.COMMUN`
- Cette fonction retourne le résultat sous la forme de listes.
- `sapply` est une fonction similaire à `lapply` mais le résultat est retourné sous forme de vecteurs, si possible.

Exemple d'utilisation de lapply

`cars` est une liste constituée de deux vecteurs `speed` et `dist`
On calcule la moyenne et les quantiles des deux composantes de la liste.

```
 cars
   speed dist
1      4    2
2      4   10
3      7    4
lapply(cars, mean)
$speed                $dist
[1] 15.4          [1] 42.98

lapply(cars, quantile, probs = (0:4)/4)
$speed
  0%  25%  50%  75% 100%
   4   12   15   19   25
$dist
  0%  25%  50%  75% 100%
   2   26   36   56  120
```

Supprimer une boucle à l'aide de la fonction `sapply`

Il est très souvent possible de supprimer les boucles `for` en utilisant `lapply` ou `sapply`.

Soit $x = (x_1, ..., x_t)$ une série représentant l'évolution du prix d'une action sur une durée de t jours.
On veut calculer M_i le prix maximum sur la période 1 à i pour $i = 1, ..., t$.

1. avec une boucle

```
M = 1:t
for ( i in 1:t)
{
M[i] = max(x[1:i])
}
```

2. sans boucle

```
FUN.max = function(n,y) max(y[1:n ])

 M  =   sapply(1:t , FUN.max ,  y = x )
```


La fonction `tapply` I

Le type `factor` est un objet vectoriel qui permet de spécifier une classification discrète (nombre fini de groupes).
La fonction `tapply` applique une fonction FUN sur les sous-groupes d'un vecteur X définis par une variable de type `factor` GRP.

`tapply(X, GRP, FUN, ...)`

```
note
[1] 10.83676 11.63757 11.07312 13.79699 10.84186
    10.72562 13.58680 14.85070 13.15659 14.36744

xgr
[1] "a" "b" "a" "b" "a" "a" "b" "a" "b" "b"

gr = factor(xgr)

gr
[1] a b a b a a b a b b
Levels: a b

tapply(note, gr, mean)
```


La fonction `tapply` II

```
       a        b
11.66561 13.30908
```

La fonction `aggregate` permet aussi d'évaluer une fonction sur des sous-ensembles. Elle ne nécessite pas la définition d'une variable de type `factor`.

Évaluer le temps de calcul

`proc.time` évalue combien de temps réel et CPU (en secondes) le processus en cours d'exécution a déjà pris.

```
ptm <- proc.time()
for (i in 1:50) mean(runif(1000))
proc.time() - ptm
utilisateur     système      écoulé
      0.016       0.001       0.027
```

Comparaison de trois programmations for/replicate/apply

```r
#boucle
  x=1:M
  for (i in 1:M)  x[i] = mean(
     runif(n))
#replicate
 x=replicate(M,mean(runif(n)))
#apply
 Z=matrix(runif(n*M) , ncol=M)
 x=apply(Z , 2, mean)
```

```
(n= 10^3 , M= 10^4)
      user  system elapsed
wfor  0.680   0.051   0.760
wrep  0.618   0.018   0.632
wapp  1.132   0.157   1.286

(n= 10^2 , M= 10^5)
      user  system elapsed
 wfor  3.160   0.239   3.451
 wrep  2.522   0.066   2.611
 wapp  3.031   0.253   3.456

(n= 10 , M= 10^6)
      user  system elapsed
wfor  32.077   0.419  43.655
wrep  37.219   0.481  41.147
wapp  29.911   0.392  30.357
```




SECTION 12

Exercices

Énoncé — Boucles

1. Avec une boucle `for`, afficher les carrés des entiers de 1 à 10.
2. Calculer la somme des entiers de 1 à 100 avec une boucle `for`, puis vérifier avec `sum(1:100)`.
3. Avec une boucle `while`, afficher les puissances de 2 tant qu'elles restent inférieures à 1000.
4. Avec une boucle `for`, compter le nombre de valeurs paires dans `x <- c(3,8,15,22,7,10)`.

Énoncé — Boucles

5. Construire la table de multiplication 1–10 sous forme d'une matrice, à l'aide de **deux boucles `for` imbriquées**.
6. Avec une boucle `while`, générer les 15 premiers termes de la suite de Fibonacci ($u_0 = 0$, $u_1 = 1$, $u_n = u_{n-1} + u_{n-2}$).

Énoncé — if / else

1. Écrire un test qui affiche `"positif"`, `"negatif"` ou `"nul"` selon le signe d'un nombre `x`.
2. Dans une boucle `for` parcourant un vecteur, utiliser `if`/`else` pour afficher si chaque valeur est `"paire"` ou `"impaire"`.
3. Écrire une fonction `signe(x)` qui retourne `-1`, `0` ou `1` selon le signe de `x`, à l'aide de `if`/`else if`/`else`.

Énoncé — if / else

4. **FizzBuzz.** Pour `i` de 1 à 30 : afficher `"Fizz"` si `i` est multiple de 3, `"Buzz"` si multiple de 5, `"FizzBuzz"` si multiple des deux, sinon `i`.
5. Écrire une fonction `est_premier(n)` qui teste si `n` est un nombre premier (boucle `for` + `if` avec un indicateur booléen).

Énoncé — apply / sapply

1. Créer `M <- matrix(1:12, ncol = 3)`. Calculer la moyenne de chaque **colonne** avec `apply`, puis de chaque **ligne**.
2. Utiliser `sapply` pour appliquer `carre(x) =` x^2 à tout le vecteur `1:10`.
3. Utiliser `sapply` pour calculer la moyenne de chaque colonne de `iris[,1:4]`.
4. Comparer `sapply` et `lapply` sur cette même question : quelle est la différence entre les deux objets retournés ?

Énoncé — apply / sapply

5. Utiliser `sapply` avec une **fonction anonyme** pour centrer-réduire chaque colonne de `iris[,1:4]` (soustraire la moyenne, diviser par l'écart-type).
6. Utiliser `apply` avec une fonction qui retourne `c(min=..., max=...)` pour chaque colonne de `M`. Que remarque-t-on sur la forme du résultat ?

Énoncé — Monte Carlo

1. Estimer π par la méthode de Monte Carlo : tirer N points uniformes dans $[-1,1]^2$ et calculer la proportion tombant dans le disque unité.
2. Estimer $\displaystyle\int_0^1 e^{-x^2}\,dx$ par Monte Carlo (moyenne de e^{-U^2} pour $U \sim \mathcal{U}(0,1)$), et comparer avec `integrate()`.
3. Calculer ces deux estimations pour $N = 100$, 1 000 et 100 000. Que peut-on dire de la précision en fonction de N ?

Énoncé — Monte Carlo

5. Estimer $P(X + Y > 1)$ pour $X, Y \sim \mathcal{U}(0,1)$ indépendantes par Monte Carlo, et comparer à la valeur théorique (aire d'un triangle).
6. Construire un intervalle de confiance approché à 95 % pour l'estimation de π obtenue en 1 (utiliser l'écart-type empirique de l'indicatrice `dans_cercle` et le TCL).

SECTION 13

Outils graphiques en statistique

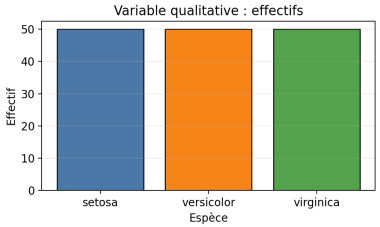
Objectif

Un graphique statistique doit être choisi selon la nature des variables.

Type de données	Question typique	Graphique de base R
Qualitative	Quelles modalités dominent ?	`barplot()`, `pie()`
Quantitative discrète	Quels effectifs par valeur ?	`plot(..., type="h")`
Quantitative continue	Quelle distribution ?	`hist()`, `density()`, `ecdf()`
Bivariée	Relation entre deux variables ?	`plot()`, `boxplot()`
Multivariée	Structure globale ?	`pairs()`, `heatmap()`

Variable qualitative : diagramme en barres

```
barplot(table(iris$Species),
        main = "Especes d'
    iris",
        xlab = "Espece",
        ylab = "Effectif")
```

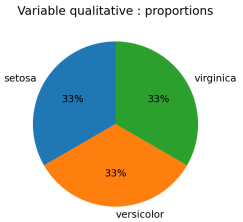


usage

Le diagramme en barres compare des effectifs ou des proportions entre modalités.

Variable qualitative : diagramme circulaire

```
pie(table(iris$Species),
    main = "Repartition")
```

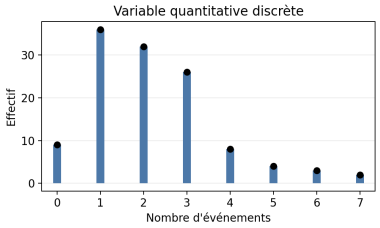


Usage

reserver pour un petit nombre de modalités
`barplot()` fournit généralement une comparaison plus précise.

Variable quantitative discrète : diagramme en bâtons

```
x <- c(0,1,2,3,2,1,2,4,5,3,2)
plot(table(x),
     type = "h",
     lwd = 5,
     xlab = "Valeur",
     ylab = "Effectif")
```

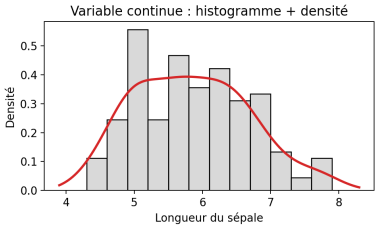


Usage

Les valeurs possibles sont isolées : le graphique ne doit pas créer artificiellement une continuité $\rightsquigarrow$ on ne trace jamais l'histogramme de variables discrètes

Variable quantitative continue : histogramme

```
hist(iris$Sepal.Length,
     breaks = 12,
     probability = TRUE,
     main = "Histogramme",
     xlab = "Longueur")
lines(density(iris$Sepal.
    Length),
      lwd = 2)
```

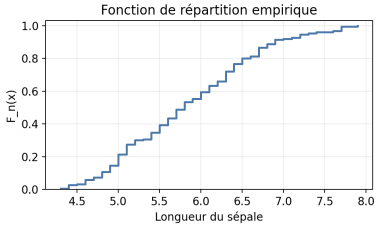


remarque

Le choix de `breaks` modifie la lecture du graphique (cours de statistique paramétrique) .

Variable quantitative continue : ECDF

```r
plot(ecdf(iris$Sepal.Length),
     verticals = TRUE,
     do.points = FALSE,
     main = "ECDF",
     xlab = "x",
     ylab = "F_n(x)")
```

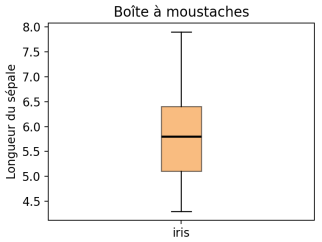


Lien avec le cours

La fonction de répartition empirique est adaptée aux lois discrète et continue

Variable quantitative continue : boîte à moustaches

```r
boxplot(iris$Sepal.Length,
        main = "Boite a
    moustaches",
        ylab = "Longueur")
```

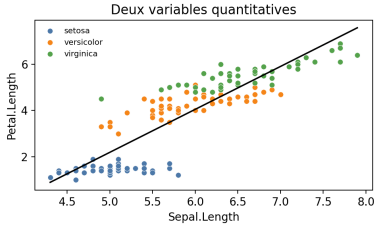


Lecture

Médiane, quartiles, dispersion et valeurs atypiques.

Deux variables quantitatives : nuage de points

```r
plot(iris$Sepal.Length,
     iris$Petal.Length,
     pch = 19,
     xlab = "Sepal.Length",
     ylab = "Petal.Length")
abline(lm(Petal.Length ~ Sepal
    .Length,
          data = iris),
       lwd = 2)
```

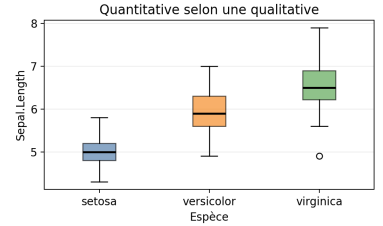


À observer

Forme, tendance, dispersion, points atypiques.

Une quantitative et une qualitative : boxplots par groupe

```r
boxplot(Sepal.Length ~ Species
    ,
        data = iris,
        xlab = "Espece",
        ylab = "Sepal.Length"
    )
```



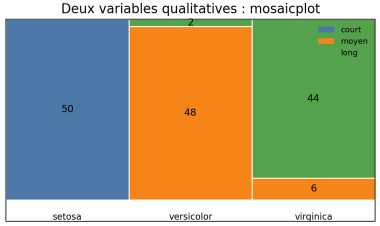
Question

Les distributions de la variable quantitative sont-elles comparables entre groupes ?

Deux variables qualitatives : tableau croisé

```r
tab <- table(var1, var2)
barplot(tab,
        beside = TRUE,
        legend = TRUE)

mosaicplot(tab,
           color = TRUE)
```

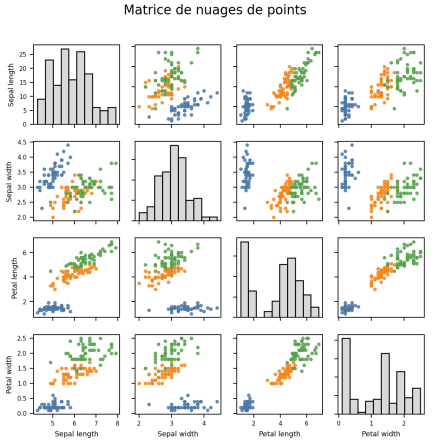


À retenir

Le `mosaicplot()` représente les effectifs d'un tableau de contingence par des surfaces.

Plusieurs variables quantitatives : matrice de nuages

```r
pairs(iris[, 1:4],
      col = as.numeric(
    iris$Species),
      pch = 19)
```

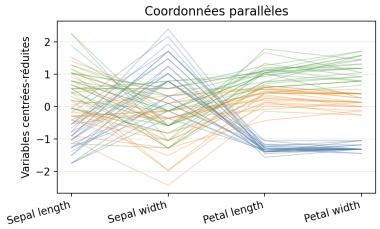


Intérêt

Repérer simultanément les relations deux à deux entre variables.

Coordonnées parallèles

```r
library(MASS)
parcoord(iris[, 1:4],
         col = as.numeric(iris
    $Species))
```

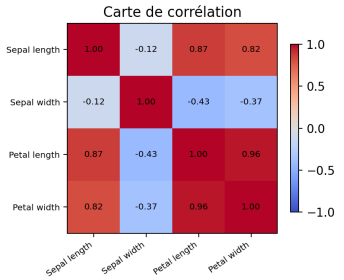


Lecture

Chaque individu est une ligne. Le graphique aide à comparer des profils multivariés.

Carte de chaleur : corrélations

```r
C <- cor(iris[, 1:4])
heatmap(C,
        symm = TRUE,
        main = "Correlations"
    )
```

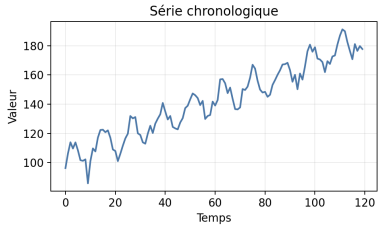


Usage

Synthèse rapide des associations linéaires entre variables quantitatives.

Cas particulier : séries chronologiques

```r
plot(AirPassengers,
     type = "l",
     xlab = "Temps",
     ylab = "Passagers",
     main = "Serie
     chronologique")
```



À observer

Tendance, saisonnalité, ruptures, valeurs atypiques.

Choisir le bon graphique

Situation	Graphique conseillé	Commande R
qualitative	Barres	`barplot(table(x))`
quantitative discrète	Bâtons	`plot(table(x), type="h")`
quantitative continue	Histogramme,	`hist()`,
	densité,	`density()`,
	processus empirique	`ecdf()`
Quantitative par groupe	Boxplots	`boxplot(y ~ g)`
Deux quantitatives	Nuage de points	`plot(x, y)`
Deux qualitatives	Barres groupées,	`barplot()`,
	mosaïque	`mosaicplot()`
Plusieurs quantitatives	Paires, heatmap	`pairs()`, `heatmap()`

conclusion

Le bon graphique dépend d'abord de la nature des variables :

qualitative, quantitative discrète, quantitative continue, bivariée ou multivariée.

En R de base, les fonctions essentielles suffisent déjà pour construire une analyse descriptive complète.

SECTION 14

Exercice

Exercice 1 – Préparation des données

On considère le jeu de données `mtcars`.

1. Charger le jeu de données.
2. Afficher les six premières observations.
3. Afficher la structure des données.
4. Identifier les variables :
   - qualitatives ;
   - quantitatives discrètes ;
   - quantitatives continues.
5. Transformer les variables `am`, `cyl` et `gear` en facteurs.

Exercice 2 – Variable qualitative

On étudie la variable `am`.

1. Calculer les effectifs de chaque modalité.
2. Représenter ces effectifs par un diagramme en barres.
3. Ajouter un titre
4. Construire ensuite un diagramme circulaire.
5. Comparer les deux représentations.

Exercice 3 – Variable quantitative continue

On étudie la variable `mpg`.

1. Calculer les statistiques descriptives.
2. Construire un histogramme.
3. Comparer deux choix du nombre de classes.
4. Ajouter une estimation de la densité.
5. Construire une boîte à moustaches.
6. Construire la fonction de répartition empirique.
7. Comparer les informations fournies par ces représentations.

Exercice 4– Deux variables quantitatives

On étudie la relation entre `wt` et `mpg`.

1. Construire le nuage de points.
2. Ajouter des titres et des labels d'axes.
3. Décrire la relation observée.
4. Calculer le coefficient de corrélation.
5. Ajouter la droite de régression.
6. Commenter la qualité de l'ajustement.

SECTION 15

Le tidyverse

Qu'est-ce que le tidyverse ?

Le **tidyverse** est un ensemble d'extensions R pour :

- importer des donnees ;
- nettoyer et transformer des tableaux ;
- produire des graphiques ;
- automatiser certaines taches ;
- rendre les analyses plus lisibles et reproductibles.

L'idee principale est d'utiliser une grammaire commune pour manipuler les donnees.

Les extensions principales

Extension	Role principal
`dplyr`	manipuler des tableaux
`ggplot2`	produire des graphiques
`tidyr`	mettre les donnees au format tidy
`readr`	importer des fichiers texte
`readxl`	importer des fichiers Excel
`stringr`	manipuler des chaines de caracteres
`forcats`	manipuler des facteurs
`purrr`	iterer proprement

Installation et chargement

Installation :

```
install.packages("tidyverse")
```

Chargement :

```
library(tidyverse)
```

Le chargement de `tidyverse` charge notamment :

- `ggplot2`, `dplyr`, `tidyr` ;
- `readr`, `stringr`, `forcats`, `purrr`.

La logique tidy

Une donnee est dite **tidy** lorsque :

1. chaque variable est une colonne ;
2. chaque observation est une ligne ;
3. chaque cellule contient une seule valeur.

Cette organisation facilite :

- les transformations avec `dplyr` ;
- les graphiques avec `ggplot2` ;
- les modèles statistiques.

Exemple : données non tidy et tidy

Format large :

id	maths	anglais
1	12	15
2	14	13

Format long :

id	matiere	note
1	maths	12
1	anglais	15
2	maths	14
2	anglais	13

Les tibbles

Le tidyverse utilise souvent des **tibbles**, proches des `data.frame`.

```
eleves <- tibble(
  nom = c("Alice", "Bob", "Chloe"),
  age = c(20, 22, 21),
  note = c(14, 11, 16)
)

> eleves
# A tibble: 3 x 3
  nom     age  note
  <chr> <dbl> <dbl>
1 Alice    20    14
2 Bob      22    11
3 Chloe    21    16
```

Avantages :

- affichage compact ;
- types des colonnes visibles ;

Le pipe |>

Le pipe passe le résultat de gauche comme *premier argument* de la fonction de droite :

`x |> f()` est équivalent à `f(x)`

```
# sans pipe : lecture de l'interieur vers l'exterieur
round(mean(sqrt(abs(x))), 2)

# avec pipe : lecture dans l'ordre des operations
x |> abs() |> sqrt() |> mean() |> round(2)
```

L'intérêt n'est pas d'écrire moins, mais de lire les étapes **dans l'ordre où elles ont lieu** — ce qui compte dès qu'une chaîne dépasse deux opérations.

Deux remarques. Le pipe `|>` est natif depuis `R` 4.1 ;

- les tutoriels en ligne utilisent souvent `%>%`, celui du paquet `magrittr`, au comportement très voisin.
- `x |> f()` exige les parenthèses : `x |> f` retourne une erreur de syntaxe.

SECTION 16

Manipuler les donnees avec dplyr

Pourquoi dplyr ?

`dplyr` fournit une grammaire simple pour manipuler des tableaux.

Les verbes principaux :

- `filter()` : choisir des lignes ;
- `select()` : choisir des colonnes ;
- `arrange()` : trier ;
- `mutate()` : créer ou modifier des variables ;
- `summarise()` : résumer ;
- `group_by()` : travailler par groupes.

Jeu de donnees exemple

On utilise ici le jeu de donnees `starwars` fourni avec le tidyverse.

```
library(tidyverse)

data <- starwars
```

Variables disponibles :

```
data |> colnames()
 [1] "name"       "height"     "mass"       "hair_color" "skin_
    color" "eye_color"
 [7] "birth_year" "sex"        "gender"     "homeworld"  "species"
       "films"
[13] "vehicles"   "starships"
```


Selectionner des colonnes : select()

```
starwars |>
  select(name, height, mass, species)
```

Selection par exclusion :

```
starwars |>
  select(-films, -vehicles, -starships)
```

Selection avec des aides :

```
starwars |>
  select(name, starts_with("h"))

#resultat
# A tibble: 87 x 4
   name               height hair_color    homeworld
   <chr>               <int> <chr>         <chr>
 1 Luke Skywalker        172 blond         Tatooine
 2 C-3PO                 167 NA            Tatooine
 3 R2-D2                  96 NA            Naboo
 4 Darth Vader           202 none          Tatooine
 5 Leia Organa           150 brown         Alderaan
```

Filtrer des lignes : filter()

```
starwars |>
  filter(species == "Human")
```

Avec plusieurs conditions :

```
starwars |>
  filter(species == "Human", height > 180)
```

Avec un ou logique :

```
starwars |>
  filter(species == "Human" | species == "Droid")
```


Trier les lignes : arrange()

Tri croissant :

```
starwars |>
  arrange(height)
```

Tri décroissant :

```
starwars |>
  arrange(desc(height))
```

On peut trier selon plusieurs variables :

```
starwars |>
  arrange(species, desc(height))
```


Créer une variable : mutate()

```
starwars |>
  mutate(height_m = height / 100)
```

Créer plusieurs variables :

```
starwars |>
  mutate(
    height_m = height / 100,
    imc = mass / height_m^2
  )
```

`mutate()` conserve le tableau initial et ajoute des colonnes.

Resumer les données : summarise()

```
starwars |>
  summarise(
    taille_moyenne = mean(height, na.rm = TRUE),
    masse_moyenne = mean(mass, na.rm = TRUE)
  )

  # A tibble: 1 x 2
  taille_moyenne masse_moyenne
           <dbl>         <dbl>
1           175.          97.3
```

Attention aux valeurs manquantes :

- `mean(height)` renvoie `NA` si la variable contient des valeurs manquantes ;
- `na.rm = TRUE` les retire du calcul.

Compter : count()

Compter les modalites d'une variable :

```
starwars |>
  count(species)
  # A tibble: 38 x 2
   species       n
   <chr>     <int>
 1 Aleena        1
 2 Besalisk      1
 3 Cerean        1
 4 Chagrian      1
 5 Clawdite      1
 6 Droid         6
 7 Dug           1
 8 Ewok          1
 9 Geonosian     1
10 Gungan        3
#  28 more rows
# Use ‘print(n = ...)‘ to see more rows
```

Trier par effectif décroissant :

```
starwars |>
  count(species, sort = TRUE)
```


Travailler par groupes : group_by()

```
starwars |>
  group_by(species) |>
  summarise(
    n = n(),
    taille_moyenne = mean(height, na.rm = TRUE)
  )

#resultat
# A tibble: 38 x 3
   species       n taille_moyenne
   <chr>     <int>          <dbl>
 1 Aleena        1             79
 2 Besalisk      1            198
 3 Cerean        1            198
 4 Chagrian      1            196
....
```

Principe :

1. on decoupe le tableau en groupes ;
2. on applique le calcul a chaque groupe ;
3. on rassemble les resultats.

Degrouper : ungroup()

Apres un `group_by()`, certaines opérations vont restées sur les groupes. Important utiliser `ungroup()` lorsque les calculs groupes sont termines.

```
res <- starwars |>
  group_by(species) |>
  summarise(n = n()) |>
  ungroup()
```


Enchainer une analyse avec le pipe

```
starwars |>
  filter(!is.na(height), !is.na(species)) |>
  group_by(species) |>
  summarise(
    n = n(),
    taille_moyenne = mean(height)
  ) |>
  filter(n >= 2) |>
  arrange(desc(taille_moyenne))
```

Le pipe donne une lecture naturelle de l'analyse.

Les erreurs frequentes avec dplyr

- oublier `na.rm = TRUE` dans les moyennes ;
- confondre = et == dans `filter()` ;
- oublier les guillemets autour des modalites texte ;
- oublier qu'un tableau reste groupe apres `group_by()` ;
- créer un objet sans l'assigner avec <- ;

Jointures : idee generale

Une jointure sert a combiner deux tableaux a partir d'une cle commune.

```
notes <- tibble(id = c(1, 2), note = c(14, 16))
eleves <- tibble(id = c(1, 2), nom = c("Alice", "Bob"))

left_join(eleves, notes, by = "id")
```

Jointures courantes :

- `left_join()` : garder toutes les lignes du premier tableau ;
- `inner_join()` : garder seulement les correspondances ;
- `full_join()` : garder toutes les lignes des deux tableaux.



SECTION 17

Mettre en ordre avec tidyr

Pourquoi tidyr ?

`tidyr` sert a transformer la structure d'un tableau.

Cas typiques :

- des variables sont stockees dans des noms de colonnes ;
- une colonne contient plusieurs informations ;
- plusieurs colonnes devraient etre rassemblees ;
- certaines combinaisons de variables sont absentes.

Passer au format long : pivot_longer()

```
notes <- tibble(
  id = c(1, 2),
  maths = c(12, 14),
  anglais = c(15, 13)
)

notes_long <- notes |>
  pivot_longer(
    cols = c(maths, anglais),
    names_to = "matiere",
    values_to = "note"
  )
```


Passer au format large : pivot_wider()

```
notes_large <- notes_long |>
  pivot_wider(
    names_from = matiere,
    values_from = note
  )
```

`pivot_longer()` et `pivot_wider()` sont deux operations inverses dans beaucoup de situations.

Separer et reunir des colonnes

Separer une colonne :

```
df |>
  separate(nom_complet, into = c("prenom", "nom"), sep = " ")
```

Reunir deux colonnes :

```
df |>
  unite(nom_complet, prenom, nom, sep = " ")
```




SECTION 18

Visualiser avec ggplot2

Objectifs

1. **Pourquoi ggplot2**
   Comprendre la logique de la grammaire des graphiques et ce qu'elle change par rapport au R de base.
2. **Construire un graphique**
   Assembler données, esthétiques (aes) et géométries (geom) avec l'opérateur +.
3. **Geoms et esthétiques**
   Nuages de points, barres, histogrammes, boîtes à moustaches — et le mapping de couleurs, formes, tailles.
4. **Petits multiples**
   Dupliquer un graphique par sous-groupe avec `facet_wrap()`.
5. **Habiller et exporter**
   Titres, thèmes, et sauvegarde avec `ggsave()`.

SECTION 19

Recoder des variables

Recodage avec case_when()

```
starwars |>
  mutate(
    classe_taille = case_when(
      height < 150 ~ "petit",
      height < 180 ~ "moyen",
      height >= 180 ~ "grand",
      TRUE ~ NA_character_
    )
  )
```

`case_when()` est pratique pour construire des categories.

Facteurs et ordre des modalites

Pour controler l'ordre d'affichage dans les tableaux et graphiques :

```
donnees <- donnees |>
  mutate(
    niveau = factor(
      niveau,
      levels = c("faible", "moyen", "eleve")
    )
  )
```

Le package `forcats` propose plusieurs fonctions utiles pour manipuler les facteurs.

Pourquoi ggplot2 plutôt que le R de base ?

plot() — R de base

- Une fonction différente pour chaque type de graphique
- Personnalisation souvent verbeuse (légendes manuelles, etc.)
- Superposer des couches est peu naturel

ggplot2

- Un seul système cohérent : la grammaire des graphiques
- On construit le graphique couche par couche avec +
- Fait partie du tidyverse, s'intègre à dplyr
- Devenu le standard de facto pour visualiser en R

La grammaire des graphiques

Un graphique = des couches qu'on empile avec l'opérateur +

`data`	le jeu de données à représenter
`aes()`	le mapping : quelles variables → x, y, couleur, ta
`geom_...()`	la représentation visuelle : points, barres, lignes.
`facet_...()`	optionnel — dupliquer en petits multiples
`labs() / thème()`	optionnel — titres, légendes, habillage

Installer et charger ggplot2

```
install.packages("ggplot2")
library(ggplot2)

# ou, en installant tout le tidyverse
      :
install.packages("tidyverse")
library(tidyverse)
```

Bon à savoir

`install.packages()` ne se fait qu'une fois par machine.

`library()` se fait à chaque nouvelle session R.

`tidyverse` charge ggplot2, dplyr, tidyr… en un seul appel.

Anatomie d'un graphique ggplot

```
ggplot(data = mpg, aes(x = displ, y = hwy)) +
  geom_point()
```

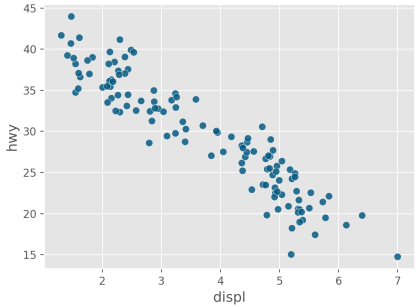
`ggplot(data = ...)`	le jeu de données utilisé
`aes(x = ..., y = ...)`	le mapping : quelles colonnes sur quels axes
`geom_point()`	la couche géométrique : comment afficher les données
`+`	l'opérateur qui empile les couches entre elles

Premier graphique : un nuage de points

```
ggplot(mpg, aes(displ,
    hwy)) +
  geom_point()
```

Chaque point représente une observation du jeu de données `mpg` : une voiture.

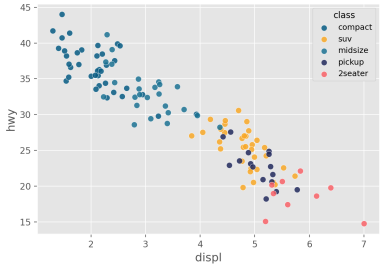
x = cylindrée du moteur (`displ`)
y = consommation autoroute (`hwy`)



Esthétique mappée ou esthétique fixe ?

```
# mappee : dependant des
    donnees
ggplot(mpg, aes(displ, hwy,
                color = class
    )) +
  geom_point()

# fixe : identique pour tous
    les points
ggplot(mpg, aes(displ, hwy))
    +
  geom_point(color = "
    steelblue")
```



Dans aes() → la couleur varie selon une colonne. **Hors aes()** → une valeur fixe pour tous les points.

Les geoms les plus courants

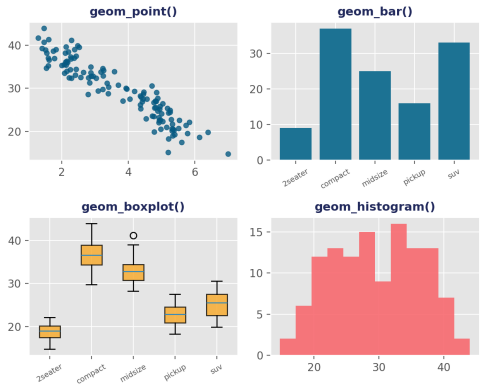
`geom_point()`
nuage de points

`geom_line()`
courbe / série temporelle

`geom_bar()`
barres (+ `geom_col()` si valeurs calculées)

`geom_boxplot()`
boîte à moustaches

`geom_histogram()`
histogramme d'une variable continue



Facettes : un graphique par sous-groupe

```
ggplot(mpg, aes(displ, hwy)
    ) +
  geom_point() +
  facet_wrap(~ class)
```

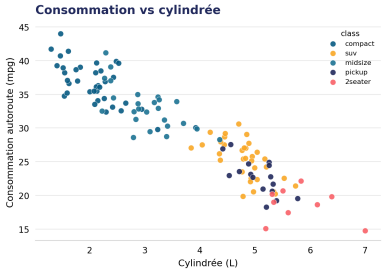


facet_wrap(~var)
un panneau par catégorie, disposés automatiquement

facet_grid(var1 ~ var2)
une grille croisant deux variables

Personnaliser : titres et thèmes

```
ggplot(mpg, aes(displ, hwy,
    color = class)) +
  geom_point() +
  labs(
    title = "Conso vs
     cylindree",
    x = "Cylindree (L)",
    y = "Consommation") +
  theme_minimal()
```



`labs()` renomme titres et axes. `theme_minimal()`, `theme_bw()`, `theme_classic()`... changent l'habillage global en une ligne.

Sauvegarder son graphique

```
ggsave("mon_graphique.png",
       width = 8, height = 5, dpi = 300)
```

`png` — Sauvegarde le dernier graphique affiché

`pdf / svg` — Formats vectoriels, idéaux pour publication

`width / height / dpi` — Contrôlent taille et résolution du fichier

Titres et thèmes

```
ggplot(starwars, aes(x = height, y = mass)) +
  geom_point() +
  labs(
    title = "Masse et taille des personnages",
    x = "Taille",
    y = "Masse"
  ) +
  theme_minimal()
```

`theme_minimal()` sert à modifier l'apparence générale du graphique, sans modifier les données représentées.

```
theme_gray()      # thème par défaut de ggplot2
theme_minimal()   # épuré, avec grille
theme_classic()   # fond blanc, axes visibles, presque pas de
    grille
theme_bw()        # fond blanc avec grille et cadre
```


La fonction ggplot

- La fonction `ggplot` est utilisée pour construire l'objet plot initial.
- elle est presque toujours suivie de `+` pour ajouter un composant au tracé.
- La fonction `aes` liste les variables utilisées dans la construction du graphique x et y ainsi que les variables qui contrôlent l'esthétique du graphique color (col dans plot) / shape (pch dans plot) / size (cex dans plot)

Il existe differentes façons d'appeler la fonction `ggplot`

1. si tous les calques utilisent les mêmes données dans l'objet data.frame `df` et les mêmes arguments de style. graphique

   ```
   ggplot (df, aes (x, y, <autres variables>))
   ```

2. si toutes les tracés utilisent les mêmes données

   ```
   ggplot (df)
   ```

3. si les calculs utilisent différents jeux de données

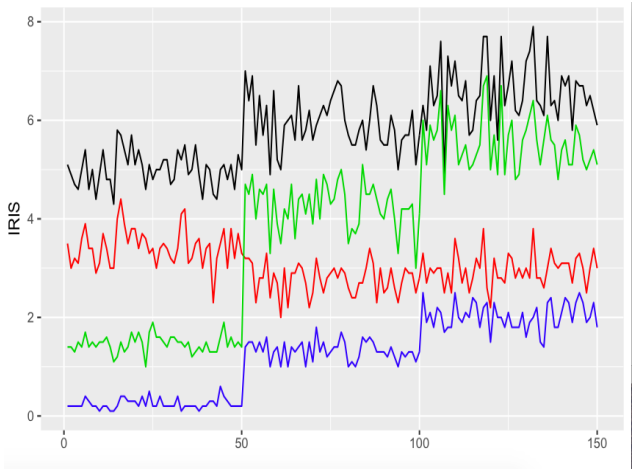
   ```
   ggplot ()
   ```

ggplot sur les données data(iris) : Equivalent du graphique avec matplot

```
data(iris)
iris2 =data.frame(x=1:150, iris)

head(iris2)
  x Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1 1          5.1         3.5          1.4         0.2  setosa
2 2          4.9         3.0          1.4         0.2  setosa
3 3          4.7         3.2          1.3         0.2  setosa
4 4          4.6         3.1          1.5         0.2  setosa
5 5          5.0         3.6          1.4         0.2  setosa
6 6          5.4         3.9          1.7         0.4  setosa


ggplot(iris2 ) +
    geom_line(aes(x,Sepal.Length),color = I(1)) +
    geom_line(aes(x, Sepal.Width),color = I(2)) +
    geom_line(aes(x, Petal.Length),color = I(3)) +
    geom_line(aes(x, Petal.Width),color = I(4))  +
    ylab("IRIS") +
    xlab("")
```

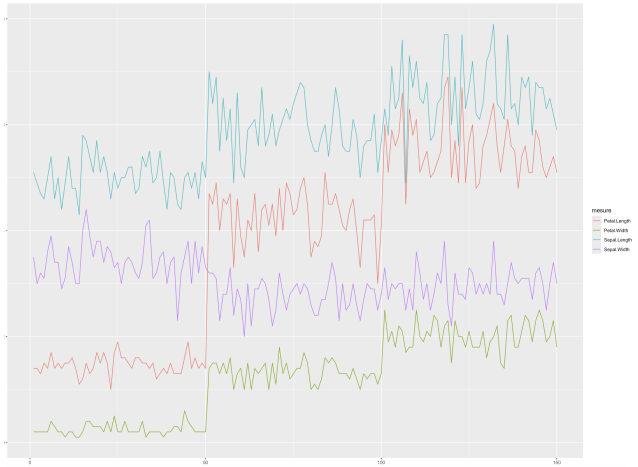



utilisation de pivot_longer

```
iris3 <- data.frame(x = 1:150, iris) |>
  tidyr::pivot_longer(Sepal.Length:Petal.Width, names_to = "mesure"
     , values_to = "valeur")

> iris3
       x   x.1 Species mesure       valeur
   <int> <int> <fct>   <chr>         <dbl>
 1     1     1 setosa  Sepal.Length    5.1
 2     1     1 setosa  Sepal.Width     3.5
 3     1     1 setosa  Petal.Length    1.4
 4     1     1 setosa  Petal.Width     0.2
 5     2     2 setosa  Sepal.Length    4.9

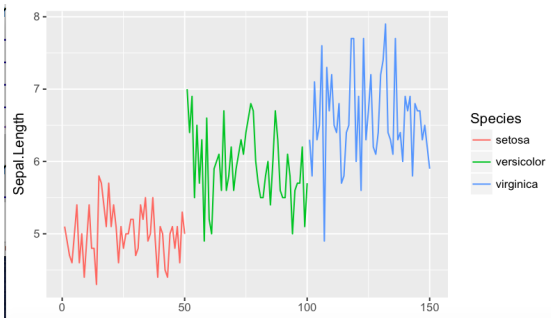
ggplot(iris3, aes(x, valeur, color = mesure)) +
  geom_line() +
  ylab("IRIS")
```

Suite données data(iris)

La couleur de la courve est données par une variable qualitative

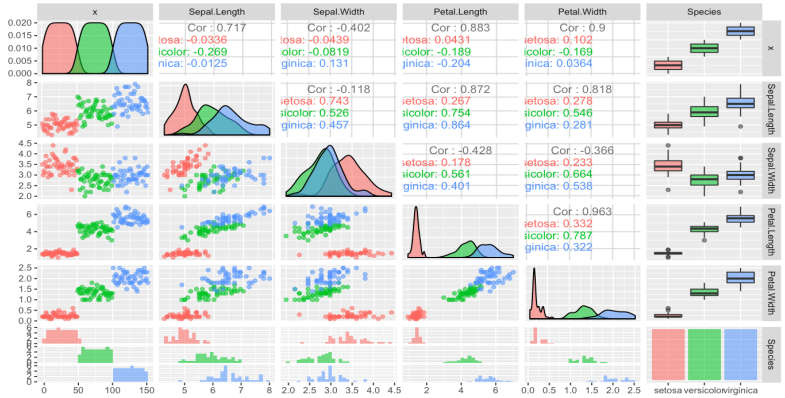
```
ggplot(iris2, aes(x,Sepal.Length, color = Species)) +
  geom_line() +
  ylab("IRIS")
```



Autre package : (GGally)

Une version sophistiquée de `pairs`

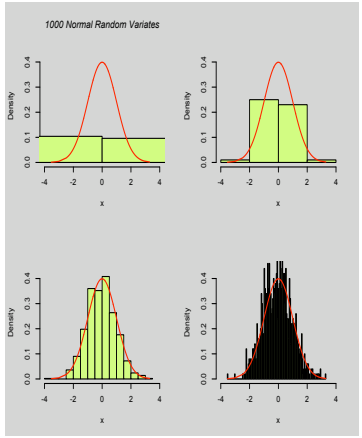
```
ggpairs(iris, aes(color = Species, alpha = 0.4))
#alpha contrôle la transparence des éléments graphiques.
```



SECTION 20

Inférence statistique

histogramme : le choix du nombre de classes



1. On simule un échantillon suivant la loi gaussienne de taille 1000
2. On trace l'histogramme pour différentes valeurs du nombre de classes.
3. On compare l'histogramme avec la densité théorique de loi gaussienne *(courbe en rouge)*

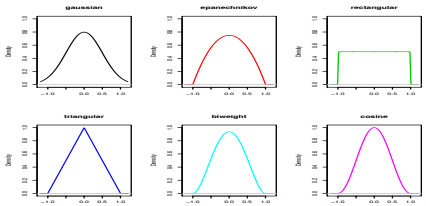
`hist(x, nclass=*** ,proba=T)` ou `hist(x,proba = T)` par défaut le nombre de classes est optimisé pour des échantillons gaussiens

Précision sur la fonction `density`

Soit $X_1, \cdots, X_n$ n variables aléatoires i.i.d. suivant la loi de densité f.
La fonction `density` calcule l'estimateur de f suivant

$$\hat{f}_n(x) = \frac{1}{n}\sum_{k=1}^{n} \frac{1}{\texttt{bw}_n} \texttt{kern}\left(\frac{x - X_k}{\texttt{bw}_n}\right)$$

- le choix du noyau `kern`

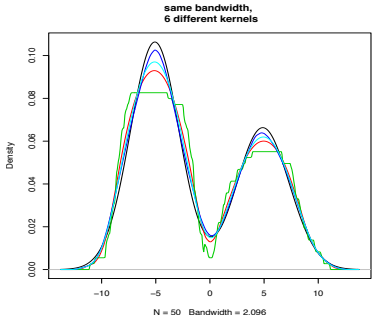


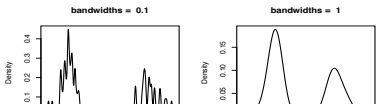
- la dimension de la fenêtre `bw`. Par défaut le paramètre est optimisé pour un échantillon gaussien

Illustration sur un mélange gaussien

On teste la fonction `density` sur des données simulées [2] suivant un mélange de lois gaussiennes :

$$f(x) = \frac{1}{4}\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(x-5)^2} + \frac{3}{4}\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(x+5)^2}$$





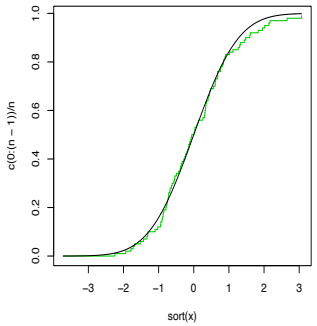
Fonction de répartition empirique (ecdf)

La fonction de répartition empirique est définie par

$$\hat{F}_n(t) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}_{]-\infty t]}(X_i) \xrightarrow[p.s.]{n\to\infty} F(t)$$

C'est un estimateur de la fonction de répartition.

```
 x= rnorm(100)
Fn=ecdf(x)
plot(Fn,col="green")
 z=seq(min(x),max(x),0.01)
 lines(z,pnorm(z), col=1,lwd=2)
```



Estimation de la moyenne : comportement asymptotique.

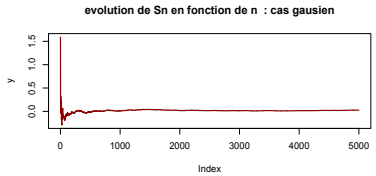
Soit $X_1, \ldots, X_n$ une suite de variables aléatoires iid, si $E|X_1| < \infty$ alors

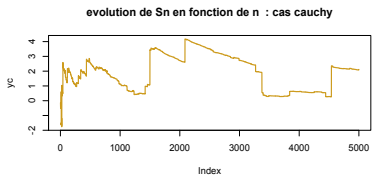
$$S_n = \frac{1}{n}\sum_{i=1}^{n} X_i \to E(X_1)$$

```
n=5000

#loi gaussienne
     x=rnorm(n,0,1)
     y=cumsum(x)/(1:n)
     plot(y, type=''l'')

# loi de  Cauchy ou Student(1
    ddl)
     xc=rt(n,1)
     yc=cumsum(xc)/(1:n)
     plot(yc, type=''l'')
```





Les fonctions pour les tests statistiques classiques :

```
t.test(x,mu=5,alt="two.sided")      #student
t.test(x,y,alt="less", conf=.95)
```

```
 var.text(x,y)                       # comparaison variance
cor.test(x,y)                       # non correlation
chisq.test(x,y)                     #indépendance
Box.test(z, lag = 1)                #non correlation
```

```
 shapiro.test(x)                     #normalité
ks.test(x,"pnorm")                  #normalité K-S
ks.test(x,y)                        # même distribution
```


Exemple : Test de Student `t.test()`

$X_1, \ldots, X_n$ iid $\mathcal{N}(1,1)$ et $Y_1, \ldots, Y_m$ iid $\mathcal{E}(1)$
Test $H_0 : E(X) = E(Y)$ vs $H_1 : E(X) \neq E(Y)$

```
 x = rnorm(100,1,1)
 y = rexp(200,1)
 t.test(x,y)

        Welch Two Sample t-test
data:  x and y
t = -0.2178, df = 178.446, p-value = 0.8278
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.2648092  0.2121608
sample estimates: mean of x : 0.9544127  mean of y : 0.9807369
```


Test d'ajustement

On construit un générateur de nombres aléatoires suivant la loi normale en utilisant le Théorème Central Limite appliqué à des variables aléatoires iid suivant la loi uniforme sur $(0,1)$.

Méthode de simulation non exacte ....

$U_1, \ldots, U_n$ iid suivant la loi uniforme

$$\sqrt{12n}(\bar{U}_n - \frac{1}{2}) \Rightarrow X \sim N(0,1) \qquad \bar{U}_n = \frac{1}{n}\sum_{i=1}^{n} U_i$$

Le générateur s'écrit

```
simU<-function(n.samp,n)
{
y = matrix(runif(n.samp*n),ncol=n)
(apply(y, 1, mean) - 1/2) * sqrt(12 * n)
}
# la fonction retourne un échantillon de longueur n.samp
```

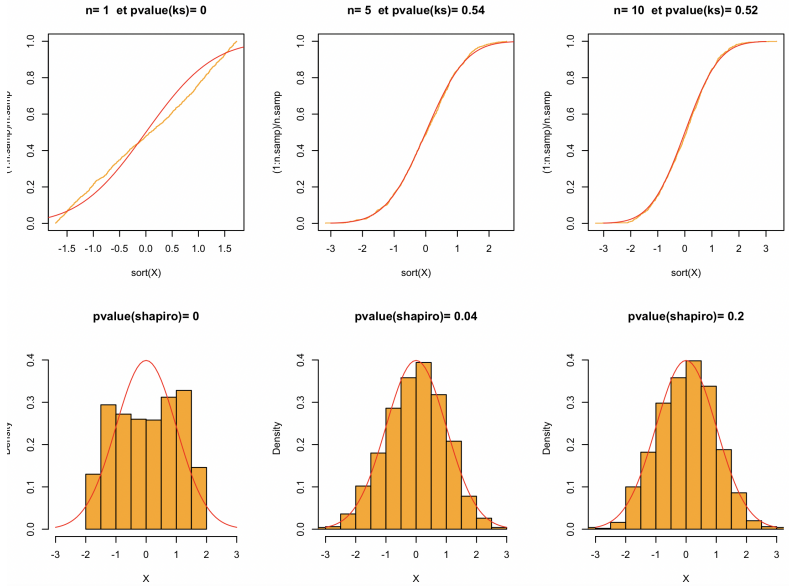

Validité de l'algorithme

Pour différentes valeurs de $n = 1,\ 5,\ 10$

1. On teste la normalité à l'aide de deux procédures de test, le test de Kolmogorov et celui de Shapiro.
2. On compare la fonction de répartition empirique et la fonction de répartition théorique.
3. On compare l'estimation de la densité par un histogramme et la densité théorique.

```
n.samp = 1000
par(mfcol = c(2, 3))
for(n in c(1,5,10))
{
X = simU(n.samp,n)
test1=ks.test(X,"pnorm",0,1)$p.value
test2 = shapiro.test(X)$p.value
plot(sort(X),(1: n.samp )/n.samp , type="s", col="orange")
lines(seq(-3,3,.1),pnorm(seq(-3,3,.1)),col="red")
title(paste("n=",n," et pvalue(ks)=",floor(c(test1)*100)/100))
hist(X,col="orange", xlim=c(-3,3), proba=T,main="", ylim=c(0,.43),
     nclass=10)
lines(seq(-3,3,.1),dnorm(seq(-3,3,.1)),col="red")
title(paste("pvalue(shapiro)=",floor(c(test2)*100)/100))
}
```

Les résultats



Régression linéaire

Le modèle le plus simple

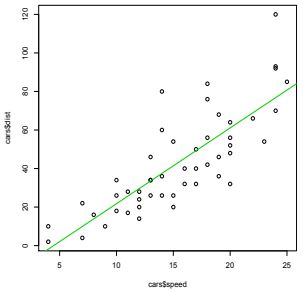
$$y = ax + b + \epsilon$$

Pour réaliser une régression linéaire, par la méthode des moindres carrés, on utilise la fonction `lm` .
Sur les données `cars`

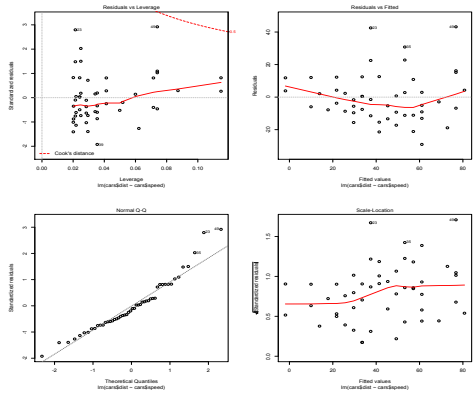
```
 lm(cars$dist~ cars$speed)
Call:
lm(formula = cars$dist ~ cars$speed)

Coefficients:
(Intercept)   cars$speed
    -17.579        3.932

 plot(cars$speed,cars$dist)
 abline(reg, col=3,lwd = 2 )
```



Visualisation : plot.lm



Remarque : Si les données sont disponibles sous la forme d'une liste on peut utiliser la syntaxe

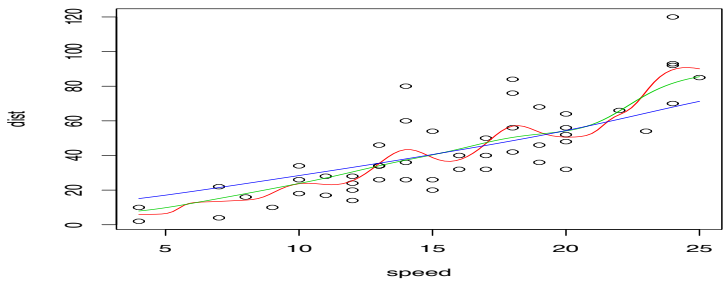
```
fit = lm(dist ~ speed, cars)
```


Extension

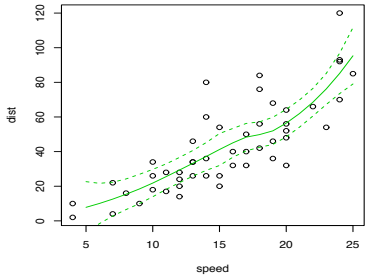
- Régression multiple `lm(v ~ v1 + v2 + v3)`
- Régression linéaire généralisée `glm`
- etc
- Méthodes non paramétriques

Régression non paramétrique

```
data(cars)
attach(cars)
plot(speed, dist)
lines(ksmooth(speed, dist, "normal", bandwidth=2), col=2)
lines(ksmooth(speed, dist, "normal", bandwidth=5), col=3)
lines(ksmooth(speed, dist, "normal", bandwidth=10), col=4)
```



Polynômes Locaux




```
data(cars)
cars.lo = loess(dist ~ speed,
     cars)
p = predict(cars.lo)
plot(cars)
lines(seq(5, 30, 1),p$fit,col
     =3)
```


SECTION 21

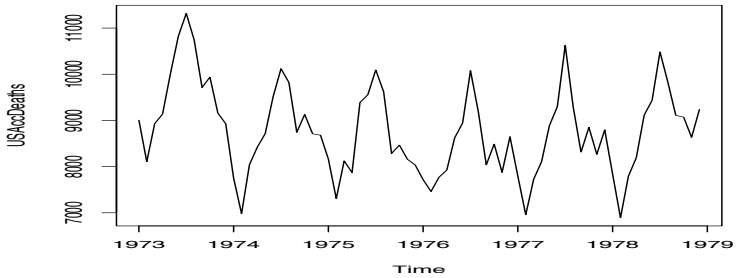
Séries Chronologiques

L'objet "série chronologique" est une liste qui contient

- les valeurs observées,
- la fréquence des observations,
- la date de la première observation
- la date de la dernière, etc...

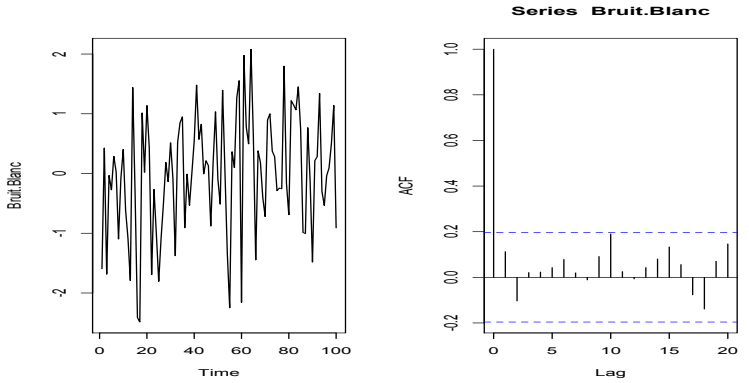
Exemple

```
 data(USAccDeaths)
 plot(USAccDeaths)
  head(USAccDeaths)
      Jan   Feb   Mar   Apr   May   Jun
1973 9007  8106  8928  9137 10017 10826
```



Pour créer une série chronologique, on utilise la fonction `ts`

```
bruit.blanc=ts(rnorm(100), frequency = 1,  start = c(1), end=c(100)
    )
plot(bruit.blanc)
acf(bruit.blanc, type="correlation")
#ou type="covariance" ou type="partial"
```



Filtre

- Premier exemple de filtre : $(I - L^p)^d$, on utilise la fonction `diff`

```
diff(x,lag=p,differences=d)
```

- La fonction `filter` permet d 'appliquer des filtres linéaires :

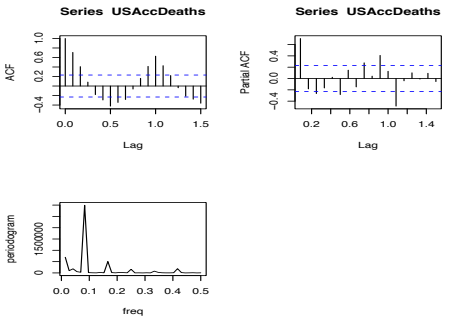
```
y=filter(x,sides= 1, method = ``convolution'' ,filter=c(2,3))
# y[i] = 2*x[i] + 3*x[i-1]
y=filter(x,sides= 2, method = ``convolution'' ,filter=c(2,3,4))
# y[i] = 2*x[i-1] + 3*x[i] +4]*x[i+1]
y=filter(x,method =''recurcive'' ,filter=c(2,3))
#y[i] = x[i] + 2*y[i-1] + 3*y[i-2]
```

Ces fonctions permettent en particulier de créer un générateur de processus ARMA.

Étude préliminaire de la série `USAccDeaths`

```
    periodogram = function(traj)
{
        n = length(traj)
        freq = 1:(n %/% 2)/(n)
        periodogram = Mod(fft(traj))[2:(n %/% 2 + 1)]^
                2/(2 * pi * n)
        plot(freq, periodogram, type = "l")
}
```

```
acf(USAccDeaths)
pacf(USAccDeaths)
periodogram(USAccDeaths)
```



modélisation et prévision AR

La fonction ar permet d'estimer les paramètres d'un modèle AR

$$A_p(L)X[t] = e[t]$$

Si l'ordre p n'est pas précisé, le meilleur modèle AR pour le critère AIC est sélectionné.

```
data(lh)
ar(lh)
Call:ar(x = lh)
Coefficients:
      1        2        3
 0.6534  -0.0636  -0.2269
Order selected 3  sigma^2 estimated as  0.1959
```



```
ar(lh, aic = FALSE, order.max = 4) # on fixe p=4
Call:
ar(x = lh, aic = FALSE, order.max = 4)
Coefficients:
      1        2        3        4
 0.6767  -0.0571  -0.2941   0.1028
Order selected 4  sigma^2 estimated as  0.1983
```

ATTENTION x doit être une série chronologique (x=ts(x))

Autour des modèles ARMA

- ARMAacf pour le calcul des covariances théorique d'un modèle ARMA
- ARMAtoMA pour le développement en MA infinie d'un modèle ARMA
- arima.sim pour simuler des trajectoires d'un modèle ARMA ou ARIMA

Modèle ARMA : Estimation

- La fonction ar permet d'estimer les paramètres d'un processus AR.
- Pour les modèles ARMA d'ordre (p,q)

$$X[t] = a[1]X[t-1]+...+a[p]X[t-p]+e[t]+b[1]e[t-1]+...+b[q]e[t-q]$$

on utilise la fonction arima, la syntaxe est
out=arima(x,order=c(p,0,q))
la sortie out est une liste contenant :
out$coef : estimation des coefficients,
out$resid : estimation des résidus e[t]
ATTENTION x doit être une série chronologique (x=ts(x))

modélisation et prévision SARIMA

Plus généralement, la fonction `arima` permet d'estimer les paramètres d'un modèle SARIMA

$$A_p(L)\alpha_P(L^s)Y[t] = \beta_Q(L^s)B_q(L)e[t] \text{ avec } Y[t] = (I-L)^d(I-L^s)^D X[t]$$

la syntaxe est la suivante :

```
out=arima(x,order=c(p,d,q),
                    seasonal=list(order=c(P,D,Q),period=s))
```

la sortie est une liste contenant :
`out$coef` : estimation des coefficients,
`out$aic` : critère AIC,
`out$resid` : estimation des résidus e[t]
option : include.mean=F ou T

Exemple

```
data(USAccDeaths)
a = c(USAccDeaths)
USAcc= ts(a[1:60],frequency=12,start=c(1973,1))
```

```
fit = arima(USAcc, order=c(0,1,1), seasonal=list(order=c(0,1,1)))
Call:
arima(x = USAcc, order = c(0, 1, 1),
                              seasonal = list(order = c(0,1, 1)))
Coefficients:
    ma1     sma1
-0.4343  -0.4419
Approx standard errors:
   ma1    sma1
0.1368  0.0122
sigma^2 estimated 114276: log likelihood = -341.73,  aic = 687.46
```

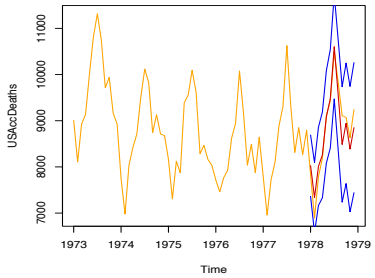

Prévision pour des modèles SARIMA

Pour prévoir à l'horizon h, on utilise la fonction `predict`

```
out=arima0(...)
 p=predict(out,h)
p$pred   #contient les prévisions
p$se     #erreurs de prévision ( écart type )
```

Exemple :

```
p = predict(fit, n.ahead=12)
plot(USAccDeaths, col="orange")
lines(p$pred,col="red3")
lines(p$pred+1.96*p$se, col="
    blue2")
lines(p$pred-1.96*p$se, col="
    blue2")
```



Plus généralement : la fonction `predict`

```
 methods(predict)
[1] "predict.ar"                "predict.arima"
[3] "predict.loess"             "predict.ppr"
[5] "predict.smooth.spline"     "predict.smooth.spline.fit"
[7] "predict.glm"               "predict.lm"
[9] "predict.mlm"
```

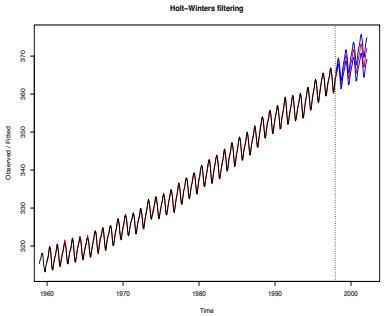
Lissage exponentiel

La méthode de lissage de Holt & Winter est disponible sous R. Les fonctions s'appelle `HoltWinters` et `predict.HoltWinters`.
Exemple : sur la série `data(co2)`

```
data(co2)
m = HoltWinters(co2)                          #lissage
p = predict(m, 50, prediction.interval = TRUE)  # prevision
plot(m, p)
```

cette fonction peut aussi être utilisée pour

- des modèles multiplicatifs
- le lissage simple si l'on impose les coefficients.



La package `forecast`

Cette librarie développée par Rob J Hyndman contient des méthodes pour l'analyse et la prévision de séries temporelles univariée

1. le lissage exponentiel, Holt Winter, ...
2. Modèles BATS (Exponential smoothing state space model with Box-Cox transformation, ARMA errors, Trend and Seasonal components)
3. modélisation automatique ARIMA.
4. modélisation automatique SARIMA .

Par défaut cette librarie utilise le traitement parallèle pour accélérer les calculs dans la sélection automatique des modèles .